

AI/BigData 인텐시브 과정

강사장철원

Section 0

코스소개

DAY1

DAY2

DAY3

DAY4

DAY5

데이터 분석을 위한 기초 수학

데이터 집계 및 시각화

DAY6

DAY7

DAY8

DAY9

DAY10

미니
프로젝트

확률분포, AB테스트

DAY11

DAY12

DAY13

DAY14

DAY15

머신러닝

DAY16

DAY17

DAY18

DAY19

DAY20

딥러닝

파이널 프로젝트

□ 사이킷런 라이브러리 소개

□ kNN

□ 회귀분석

□ 나이브 베이즈

□ 의사결정나무

□ 서포트벡터머신

AI/BigData 인텐시브 과정

Section 5. 나이브베이즈

Section 5-1. 나이브베이즈개념

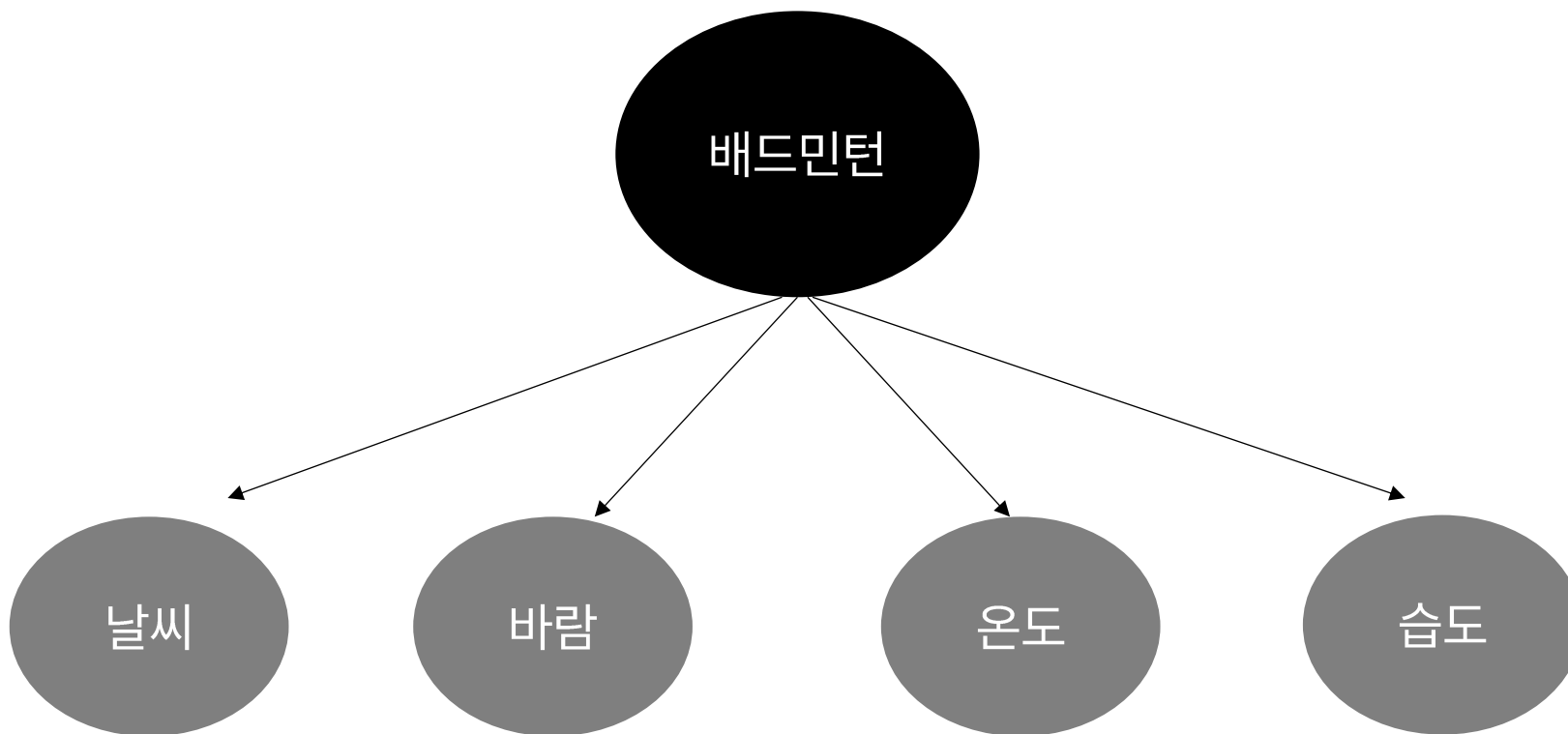
배드민턴 플레이 여부

날씨	바람	온도	습도	배드민턴
맑음	강함	추움	보통	아니오
흐림	강함	더움	보통	아니오
맑음	약함	더움	보통	네
비	약함	보통	높음	네
맑음	약함	보통	낮음	네
흐림	약함	추움	높음	아니오
흐림	약함	보통	높음	아니오
맑음	강함	더움	낮음	아니오

나이브 베이지스

피쳐는 조건부 독립(conditional independence)

$$P(X_1, X_2 | Y) = P(X_1 | Y)P(X_2 | Y)$$

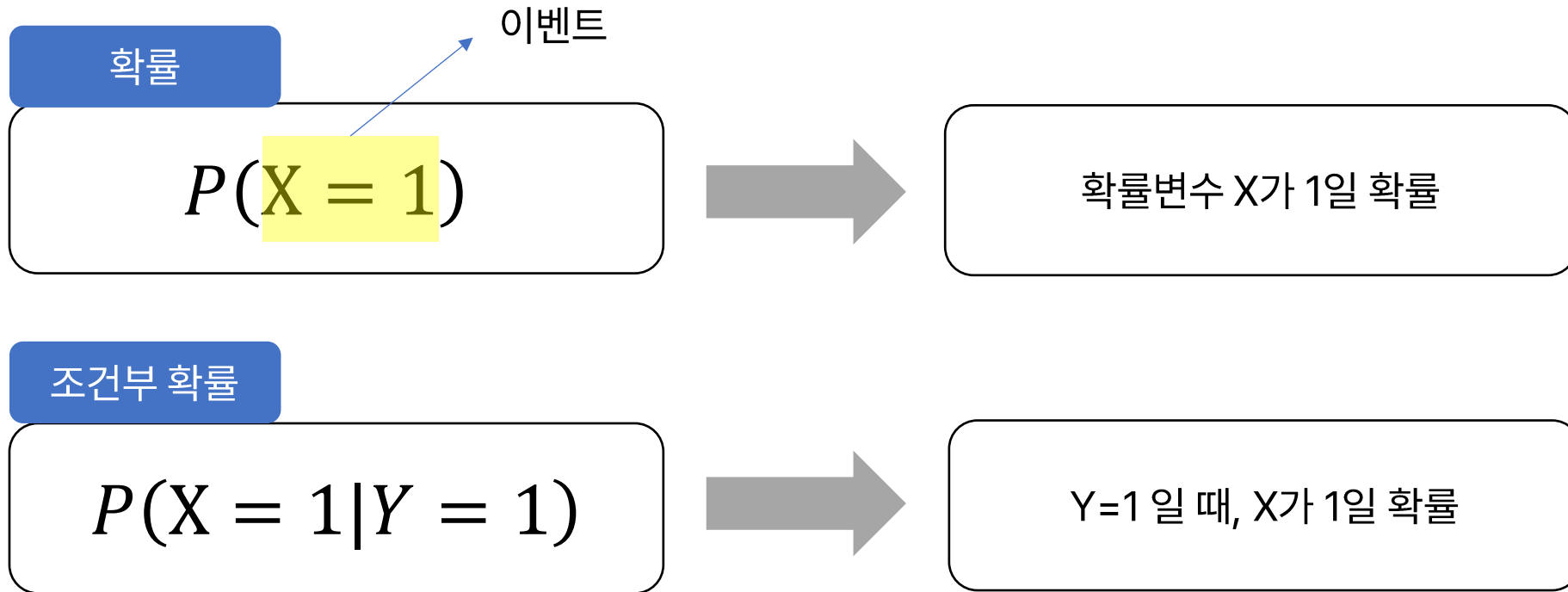


AI/BigData 인텐시브 과정

Section 5. 나이브베이즈

Section 5-2. 베이즈정리

조건부 확률(conditional probability)



조건부 확률(conditional probability)



확률변수 →

X

Y

승리

선제골 넣음

패배

선제골 먹힘

조건부 확률(conditional probability)

$$P(X|Y) = \frac{P(X \cap Y)}{P(Y)}$$

$$P(X \cap Y) = P(X|Y)P(Y) = P(Y|X)P(X)$$

$$P(X|Y) = \frac{P(Y|X)P(X)}{P(Y)}$$

사전확률(Prior Probability) : 이벤트 발생 전 확률

$$P(X|Y) = \frac{P(Y|X)P(X)}{P(Y)}$$

사후확률(Posterior Probability) : 이벤트 발생 후 확률

$$P(X|Y) = \frac{P(Y|X)P(X)}{P(Y)}$$

확률변수가 아닌 상수로 고려!
(조건이 주어졌다(given))

$$\propto P(Y|X)P(X)$$

AI/BigData 인텐시브 과정

Section 5. 나이브 베이즈

Section 5-3. 나이브 베이즈 알고리즘

배드민턴 플레이 여부

날씨	바람	온도	습도	배드민턴
맑음	강함	추움	보통	아니오
흐림	강함	더움	보통	아니오
맑음	약함	더움	보통	네
비	약함	보통	높음	네
맑음	약함	보통	낮음	네
흐림	약함	추움	높음	아니오
흐림	약함	보통	높음	아니오
맑음	강함	더움	낮음	아니오

날씨에 따른 배드민턴 플레이 여부

		배드민턴		합계
		네	아니오	
날씨	맑음	$P(\text{배드민턴}=\text{예} \mid \text{날씨}=\text{맑음})$	$P(\text{배드민턴}=\text{아니오} \mid \text{날씨}=\text{맑음})$	$P(\text{날씨}=\text{맑음})$
	흐림	$P(\text{배드민턴}=\text{예} \mid \text{날씨}=\text{흐림})$	$P(\text{배드민턴}=\text{아니오} \mid \text{날씨}=\text{흐림})$	$P(\text{날씨}=\text{흐림})$
	비	$P(\text{배드민턴}=\text{예} \mid \text{날씨}=\text{비})$	$P(\text{배드민턴}=\text{아니오} \mid \text{날씨}=\text{비})$	$P(\text{날씨}=\text{비})$
합계		$P(\text{배드민턴}=\text{예})$	$P(\text{배드민턴}=\text{아니오})$	1

날씨에 따른 배드민턴 플레이 여부

		날씨			합계
		맑음	흐림	비	
배드민턴	네	P(배드민턴=예 날씨=맑음)	P(배드민턴=예 날씨=흐림)	P(배드민턴=예 날씨=비)	P(배드민턴=예)
	아니오	P(배드민턴=아니오 날씨=맑음)	P(배드민턴=아니오 날씨=흐림)	P(배드민턴=아니오 날씨=비)	P(배드민턴=아니오)
합계		P(날씨=맑음)	P(날씨=흐림)	P(날씨=비)	1

배드민턴 플레이 여부

날씨	바람	온도	습도	배드민턴
맑음	강함	추움	보통	아니오
흐림	강함	더움	보통	아니오
맑음	약함	더움	보통	네
비	약함	보통	높음	네
맑음	약함	보통	낮음	네
흐림	약함	추움	높음	아니오
흐림	약함	보통	높음	아니오
맑음	강함	더움	낮음	아니오

배드민턴 플레이 여부

		배드민턴	
		네	아니오
날씨	맑음	2	2
	흐림	0	3
	비	1	0

		배드민턴	
		네	아니오
날씨	맑음	2/4	2/4
	흐림	0/3	3/3
	비	1/1	0/1

배드민턴 플레이 여부

날씨	바람	온도	습도	배드민턴
맑음	강함	추움	보통	아니오
흐림	강함	더움	보통	아니오
맑음	약함	더움	보통	네
비	약함	보통	높음	네
맑음	약함	보통	낮음	네
흐림	약함	추움	높음	아니오
흐림	약함	보통	높음	아니오
맑음	강함	더움	낮음	아니오

배드민턴 플레이 여부

		배드민턴	
		네	아니오
날씨	맑음	2	2
	흐림	0	3
	비	1	0
바람	강함	0	3
	약함	3	2

배드민턴 플레이 여부

		배드민턴	
		네	아니오
날씨	맑음	2/4	2/4
	흐림	0/3	3/3
	비	1/1	0/1
바람	강함	0/3	3/3
	약함	3/5	2/5

배드민턴 플레이 여부

날씨	바람	온도	습도	배드민턴
맑음	강함	추움	보통	아니오
흐림	강함	더움	보통	아니오
맑음	약함	더움	보통	네
비	약함	보통	높음	네
맑음	약함	보통	낮음	네
흐림	약함	추움	높음	아니오
흐림	약함	보통	높음	아니오
맑음	강함	더움	낮음	아니오

Section

나이브베이지스알고리즘

		배드민턴	
		네	아니오
날씨	맑음	2	2
	흐림	0	3
	비	1	0
바람	강함	0	3
	약함	3	2
온도	더움	1	2
	보통	2	1
	추움	0	2
습도	높음	1	2
	보통	1	2
	낮음	1	1

Section

나이프베이즈알고리즘

		배드민턴	
		네	아니오
날씨	맑음	2/4	2/4
	흐림	0/3	3/3
	비	1/1	0/1
바람	강함	0/3	3/3
	약함	3/5	2/5
온도	더움	1/3	2/3
	보통	2/3	1/3
	추움	0/2	2/2
습도	높음	1/3	2/3
	보통	1/3	2/3
	낮음	1/2	1/2

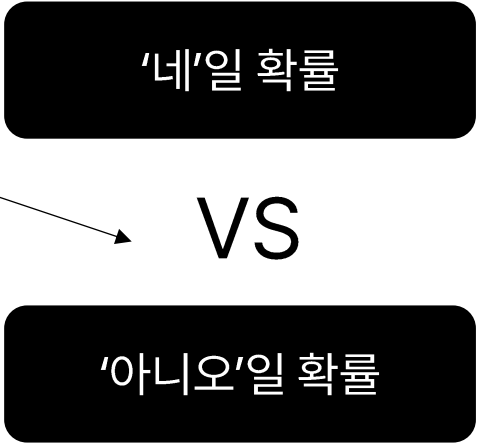
배드민턴 플레이 여부

날씨	바람	온도	습도	배드민턴
맑음	강함	추움	보통	아니오
흐림	강함	더움	보통	아니오
맑음	약함	더움	보통	네
비	약함	보통	높음	네
맑음	약함	보통	낮음	네
흐림	약함	추움	높음	아니오
흐림	약함	보통	높음	아니오
맑음	강함	더움	낮음	아니오

Section

나이브베이지스알고리즘

날씨	바람	온도	습도	배드민턴
맑음	강함	추움	보통	
흐림	강함	더움	보통	
맑음	약함	더움	보통	
비	약함	보통	높음	
맑음	약함	보통	낮음	
흐림	약함	추움	높음	
흐림	약함	보통	높음	
맑음	강함	더움	낮음	



Section

나이프베이즈알고리즘

날씨	바람	온도	습도	배드민턴
맑음	강함	추움	보통	P(아니오 날씨=맑음, 바람=강함, 온도=추움, 습도=보통)
흐림	강함	더움	보통	P(아니오 날씨=흐림, 바람=강함, 온도=더움, 습도=보통)
맑음	약함	더움	보통	P(네 날씨=맑음, 바람=약함, 온도=더움, 습도=보통)
비	약함	보통	높음	P(네 날씨=비, 바람=약함, 온도=보통, 습도=높음)
맑음	약함	보통	낮음	P(네 날씨=맑음, 바람=약함, 온도=보통, 습도=낮음)
흐림	약함	추움	높음	P(아니오 날씨=흐름, 바람=약함, 온도=추움, 습도=높음)
흐림	약함	보통	높음	P(아니오 날씨=흐림, 바람=약함, 온도=보통, 습도=높음)
맑음	강함	더움	낮음	P(아니오 날씨=맑음, 바람=강함, 온도=더움, 습도=낮음)

배드민턴 플레이 여부

$$P(X|Y) = \frac{P(X \cap Y)}{P(Y)}$$

$$P(X \cap Y) = P(X|Y)P(Y) = P(Y|X)P(X)$$

$$P(X|Y) = \frac{P(Y|X)P(X)}{P(Y)}$$

$$P(\text{아니오} | \text{날씨} = \text{맑음}, \text{바람} = \text{강함}, \text{온도} = \text{추움}, \text{습도} = \text{보통})$$



$$P(X|Y) = \frac{P(X \cap Y)}{P(Y)} \text{ 이용}$$

$$\frac{P(\text{아니오}, \text{날씨} = \text{맑음}, \text{바람} = \text{강함}, \text{온도} = \text{추움}, \text{습도} = \text{보통})}{P(\text{날씨} = \text{맑음}, \text{바람} = \text{강함}, \text{온도} = \text{추움}, \text{습도} = \text{보통})}$$



$$P(X \cap Y) = P(Y|X)P(X) \text{ 이용}$$

$$\frac{P(\text{날씨} = \text{맑음}, \text{바람} = \text{강함}, \text{온도} = \text{추움}, \text{습도} = \text{보통} | \text{아니오}) P(\text{아니오})}{P(\text{날씨} = \text{맑음}, \text{바람} = \text{강함}, \text{온도} = \text{추움}, \text{습도} = \text{보통})}$$

배드민턴 플레이 여부

$$\frac{P(\text{날씨} = \text{맑음}, \text{바람} = \text{강함}, \text{온도} = \text{추움}, \text{습도} = \text{보통} | \text{아니오}) P(\text{아니오})}{P(\text{날씨} = \text{맑음}, \text{바람} = \text{강함}, \text{온도} = \text{추움}, \text{습도} = \text{보통})}$$

조건부 독립



$$P(X_1, X_2 | Y) = P(X_1 | Y) P(X_2 | Y)$$

$$\frac{P(\text{날씨} = \text{맑음} | \text{아니오}) \times P(\text{바람} = \text{강함} | \text{아니오}) \times P(\text{온도} = \text{추움} | \text{아니오}) \times P(\text{습도} = \text{보통} | \text{아니오}) \times P(\text{아니오})}{P(\text{날씨} = \text{맑음}, \text{바람} = \text{강함}, \text{온도} = \text{추움}, \text{습도} = \text{보통})}$$

$$P(\text{아니오} | \text{날씨} = \text{맑음}, \text{바람} = \text{강함}, \text{온도} = \text{추움}, \text{습도} = \text{보통})$$

=

$$\frac{P(\text{날씨} = \text{맑음} | \text{아니오}) \times P(\text{바람} = \text{강함} | \text{아니오}) \times P(\text{온도} = \text{추움} | \text{아니오}) \times P(\text{습도} = \text{보통} | \text{아니오}) \times P(\text{아니오})}{P(\text{날씨} = \text{맑음}, \text{바람} = \text{강함}, \text{온도} = \text{추움}, \text{습도} = \text{보통})}$$

$\propto$

확률변수가 아닌 상수로 고려!
(조건이 주어졌다(given))

$$P(\text{날씨} = \text{맑음} | \text{아니오}) \times P(\text{바람} = \text{강함} | \text{아니오}) \times P(\text{온도} = \text{추움} | \text{아니오}) \times P(\text{습도} = \text{보통} | \text{아니오}) \times P(\text{아니오})$$

$$P(\text{네} | \text{날씨} = \text{맑음}, \text{바람} = \text{강함}, \text{온도} = \text{추움}, \text{습도} = \text{보통})$$



$$\frac{P(\text{날씨} = \text{맑음} | \text{네}) \times P(\text{바람} = \text{강함} | \text{네}) \times P(\text{온도} = \text{추움} | \text{네}) \times P(\text{습도} = \text{보통} | \text{네}) \times P(\text{네})}{P(\text{날씨} = \text{맑음}, \text{바람} = \text{강함}, \text{온도} = \text{추움}, \text{습도} = \text{보통})}$$



$$P(\text{날씨} = \text{맑음} | \text{네}) \times P(\text{바람} = \text{강함} | \text{네}) \times P(\text{온도} = \text{추움} | \text{네}) \times P(\text{습도} = \text{보통} | \text{네}) \times P(\text{네})$$

날씨	바람	온도	습도	배드민턴
맑음	강함	추움	보통	

$$P(\text{아니오} | \text{날씨} = \text{맑음}, \text{바람} = \text{강함}, \text{온도} = \text{추움}, \text{습도} = \text{보통})$$

VS

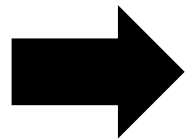
$$P(\text{네} | \text{날씨} = \text{맑음}, \text{바람} = \text{강함}, \text{온도} = \text{추움}, \text{습도} = \text{보통})$$

날씨	바람	온도	습도	배드민턴
맑음	강함	추움	보통	

$$P(\text{아니오}) \times P(\text{날씨} = \text{맑음} | \text{아니오}) \times P(\text{바람} = \text{강함} | \text{아니오}) \times P(\text{온도} = \text{추움} | \text{아니오}) \times P(\text{습도} = \text{보통} | \text{아니오})$$

VS

$$P(\text{네}) \times P(\text{날씨} = \text{맑음} | \text{네}) \times P(\text{바람} = \text{강함} | \text{네}) \times P(\text{온도} = \text{추움} | \text{네}) \times P(\text{습도} = \text{보통} | \text{네})$$



확률이 큰 쪽으로 판별

$$f_X(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

피처가 연속형 숫자인 경우 -> 가우시안 나이브 베이지스

	기온	배드민턴
평균 μ_1 표준편차 σ_1	27	예
	28	예
	26	예
평균 μ_2 표준편차 σ_2	30	아니오
	32	아니오
	31	아니오

$$P(X = 29 | \text{예}) = \frac{1}{\sqrt{2\pi}\sigma_1} e^{-\frac{1}{2}\left(\frac{x-\mu_1}{\sigma_1}\right)^2}$$

$$\rightarrow P(\text{예} | X = 29) \propto P(X = 29 | \text{예})P(\text{예})$$

$$P(X = 29 | \text{아니오}) = \frac{1}{\sqrt{2\pi}\sigma_2} e^{-\frac{1}{2}\left(\frac{x-\mu_2}{\sigma_2}\right)^2}$$

$$\rightarrow P(\text{아니오} | X = 29) \propto P(X = 29 | \text{아니오})P(\text{아니오})$$

AI/BigData 인텐시브 과정

Section 5. 나이브베이즈

Section 5-4. 나이브베이즈 분류

나이프 베이즈 분류

```
In [1]: 1 import pandas as pd
        2 df = pd.read_csv("../data/wine_data.csv")
```

```
In [2]: 1 df
```

Out[2]:

Alcohol	Malic	Ash	Alcalinity	Magesium	Phenols	Flavanoids	Nonflavanoids	Proanthocyanins	Color	Hue	Dilution	Proline	class
14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065	0
13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050	0
13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185	0
14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480	0
13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
13.71	5.65	2.45	20.5	95	1.68	0.61	0.52	1.06	7.70	0.64	1.74	740	2
13.40	3.91	2.48	23.0	102	1.80	0.75	0.43	1.41	7.30	0.70	1.56	750	2
13.27	4.28	2.26	20.0	120	1.59	0.69	0.43	1.35	10.20	0.59	1.56	835	2
13.17	2.59	2.37	20.0	120	1.65	0.68	0.53	1.46	9.30	0.60	1.62	840	2
14.13	4.10	2.74	24.5	96	2.05	0.76	0.56	1.35	9.20	0.61	1.60	560	2

rows × 14 columns

나이프 베이즈 분류

```
In [3]: ▶ 1 features = ['Alcohol', 'Malic', 'Ash', 'Alcalinity', 'Magesium', 'Phenols', 'Flavanoids',  
2             'Nonflavanoids', 'Proanthocyanins', 'Color', 'Hue', 'Dilution', 'Proline']  
3  
4 X = df[features]  
5 y = df['class']
```

나이프 베이즈 분류

In [4]: ▶

```
1 # 트레이닝/테스트 데이터 분할
2 from sklearn.model_selection import train_test_split
3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

In [5]: ▶

```
1 #데이터 표준화
2 from sklearn.preprocessing import StandardScaler
3 std_scale = StandardScaler()
4 std_scale.fit(X_tn)
5 X_tn_std = std_scale.transform(X_tn)
6 X_te_std = std_scale.transform(X_te)
```

나이브 베이지스 분류

In [6]: ▶

```
1 # 나이브 베이지스 학습
2 from sklearn.naive_bayes import GaussianNB
3 clf_gnb = GaussianNB()
4 clf_gnb.fit(X_tn_std, y_tn)
```

Out[6]:

```
▼ GaussianNB
GaussianNB()
```

In [7]: ▶

```
1 # 예측
2 pred_gnb = clf_gnb.predict(X_te_std)
3 print(pred_gnb)
```

```
[0 2 1 0 1 1 0 2 1 1 2 2 0 0 2 1 0 0 2 0 0 0 0 1 1 1 1 1 1 2 0 0 1 0 0 0 2
 1 1 2 0 0 1 1 1]
```


나이브 베이지 분류

In [8]: ▶

```
1 # 정확도(accuracy)
2 from sklearn.metrics import accuracy_score
3 accuracy = accuracy_score(y_te, pred_gnb)
4 print(accuracy)
```

0.9333333333333333

In [9]: ▶

```
1 # 정밀도(precision)
2 from sklearn.metrics import precision_score
3 precision = precision_score(y_te, pred_gnb, average='macro')
4 print(precision)
```

0.9259259259259259

In [10]: ▶

```
1 # 리콜
2 from sklearn.metrics import recall_score
3 recall = recall_score(y_te, pred_gnb, average='macro')
4 print(recall)
```

0.9523809523809524

In [11]: ▶

```
1 # f1 스코어
2 from sklearn.metrics import f1_score
3 f1 = f1_score(y_te, pred_gnb, average='macro')
4 print(f1)
```

0.9351432880844645

나이브 베이지스 분류

In [12]: ▶

```
1 # confusion matrix 확인
2 from sklearn.metrics import confusion_matrix
3 conf_matrix = confusion_matrix(y_te, pred_gnb)
4 print(conf_matrix)
```

```
[[16  0  0]
 [ 2 18  1]
 [ 0  0  8]]
```

In [13]: ▶

```
1 # 분류 레포트 확인
2 from sklearn.metrics import classification_report
3 class_report = classification_report(y_te, pred_gnb)
4 print(class_report)
```

	precision	recall	f1-score	support
0	0.89	1.00	0.94	16
1	1.00	0.86	0.92	21
2	0.89	1.00	0.94	8
accuracy			0.93	45
macro avg	0.93	0.95	0.94	45
weighted avg	0.94	0.93	0.93	45

AI/BigData 인텐시브 과정

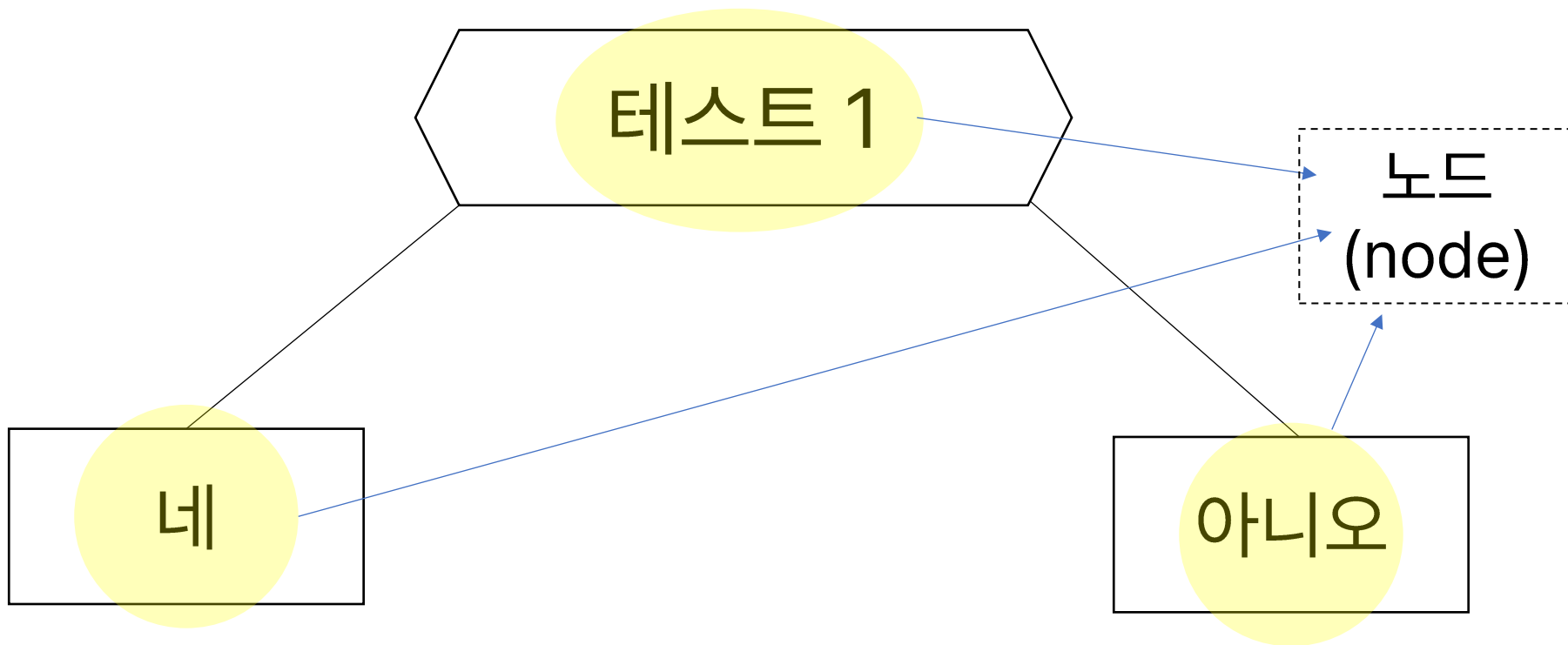
Section 6. 의사결정나무

Section 6-1. 의사결정나무 개념

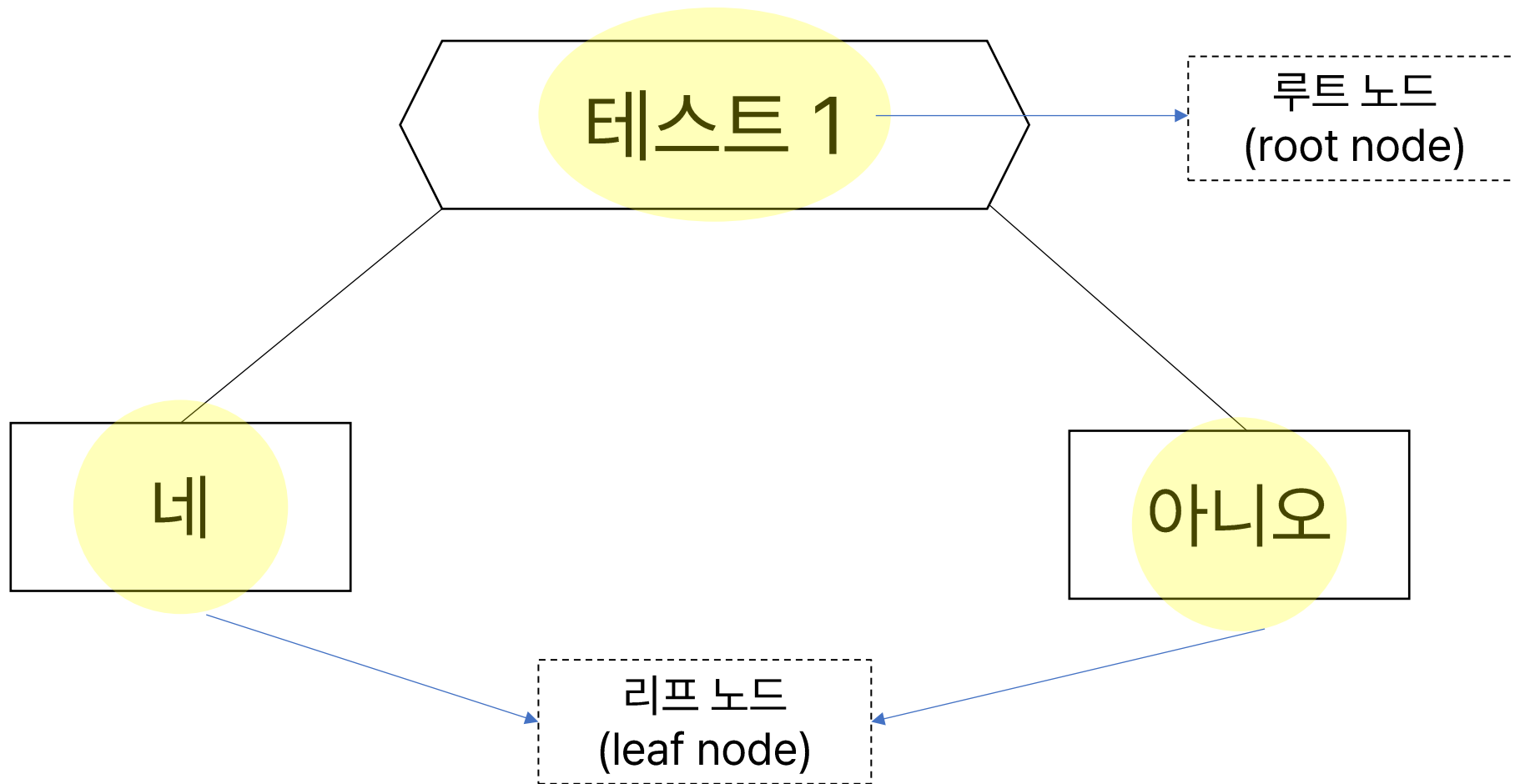
배드민턴 플레이 여부

날씨	바람	온도	습도	배드민턴
맑음	강함	추움	보통	아니오
흐림	강함	더움	보통	아니오
맑음	약함	더움	보통	네
비	약함	보통	높음	네
맑음	약함	보통	낮음	네
흐림	약함	추움	높음	아니오
흐림	약함	보통	높음	아니오
맑음	강함	더움	낮음	아니오

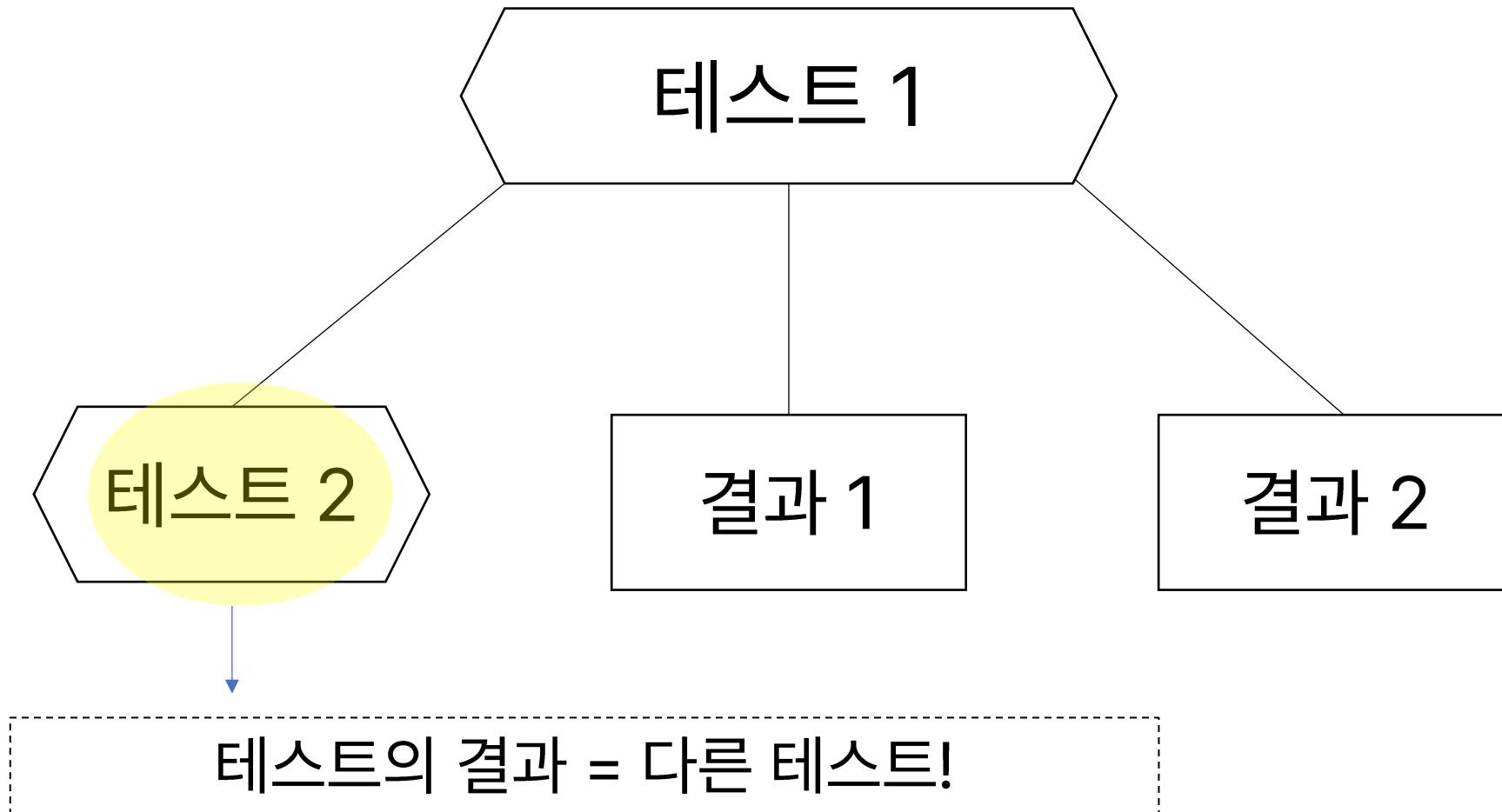
배드민턴 플레이 여부



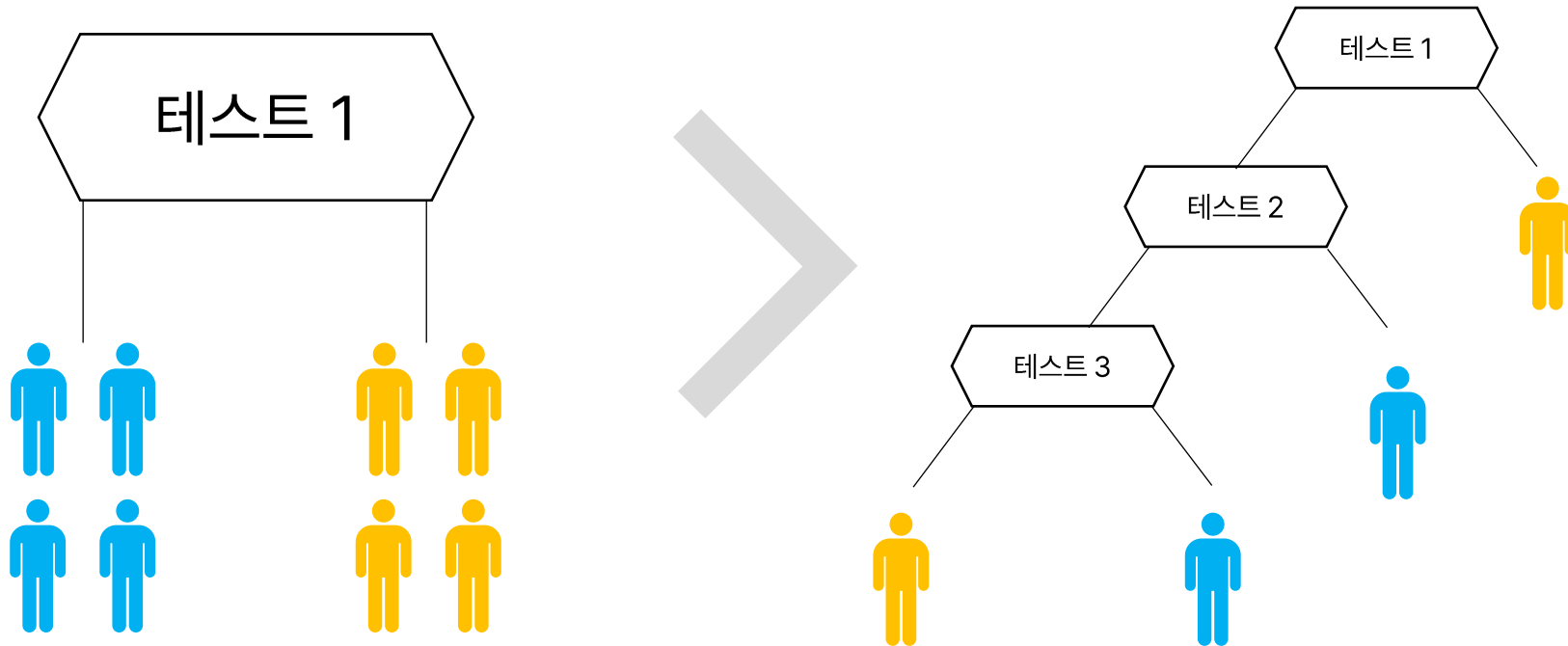
배드민턴 플레이 여부



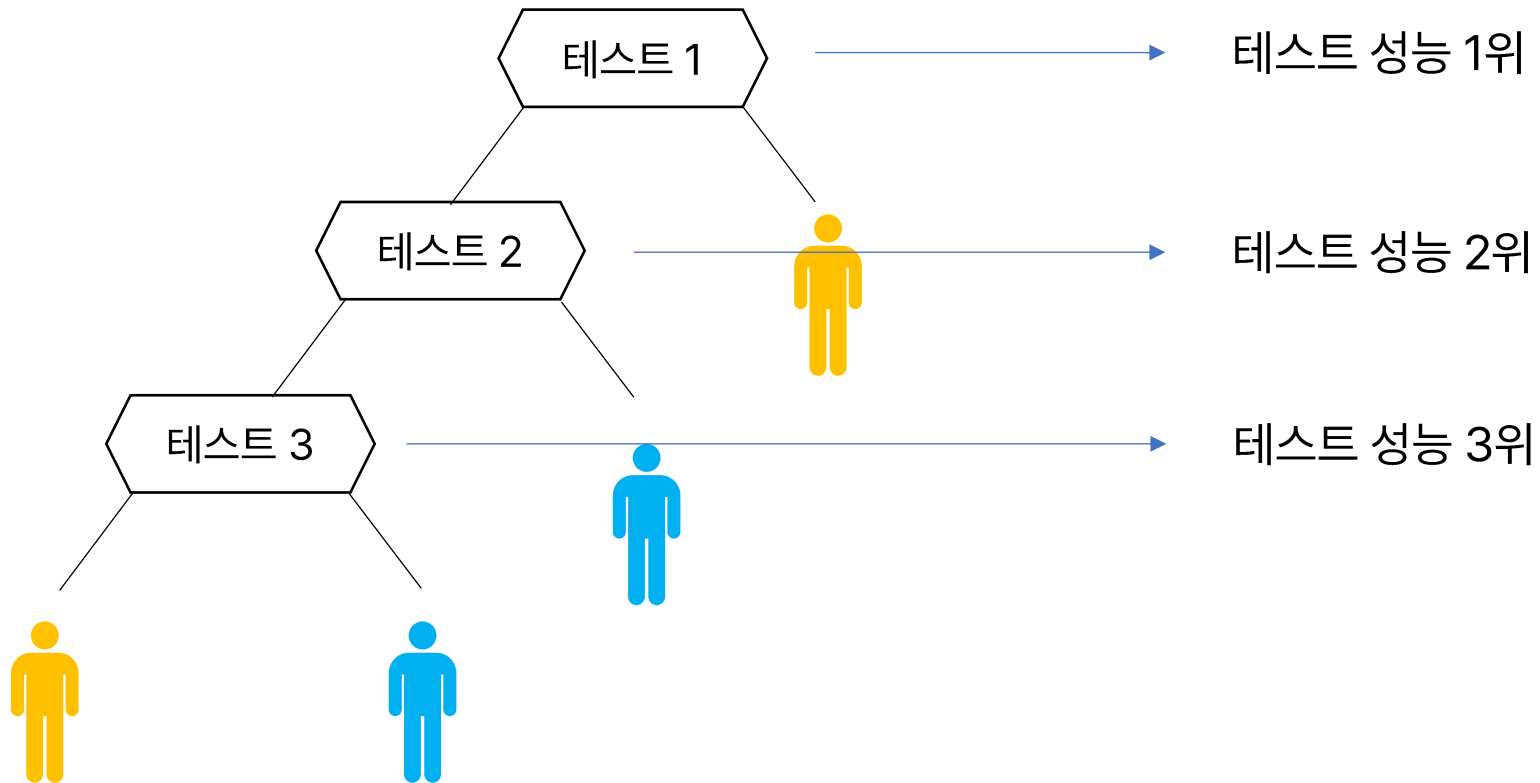
배드민턴 플레이 여부



높은 성능의 의사결정나무란?



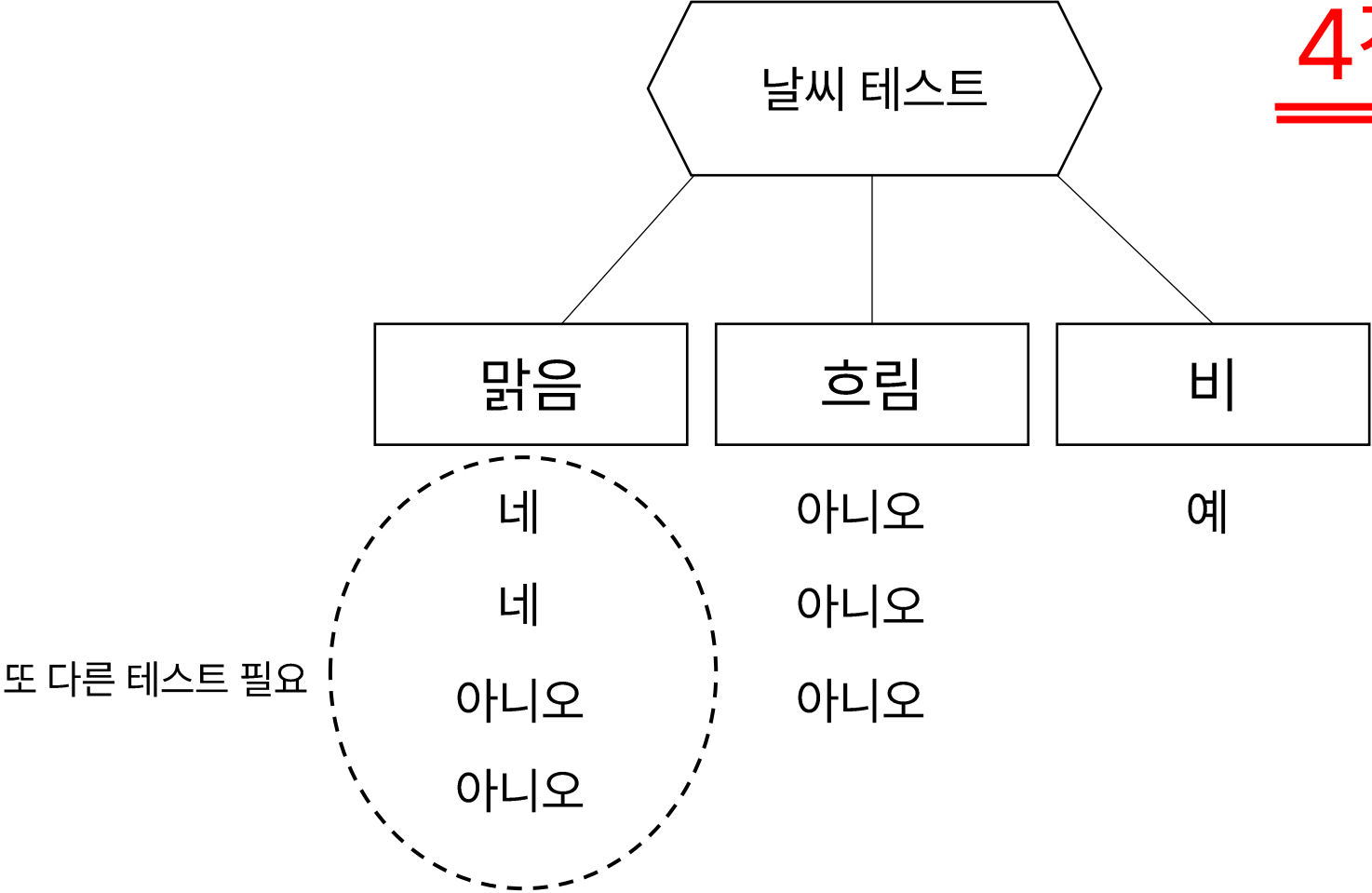
테스트 선정 순서: 성능 좋은 테스트부터 우선 선정



배드민턴 플레이 여부

날씨	바람	온도	습도	배드민턴
맑음	강함	추움	보통	아니오
흐림	강함	더움	보통	아니오
맑음	약함	더움	보통	네
비	약함	보통	높음	네
맑음	약함	보통	낮음	네
흐림	약함	추움	높음	아니오
흐림	약함	보통	높음	아니오
맑음	강함	더움	낮음	아니오

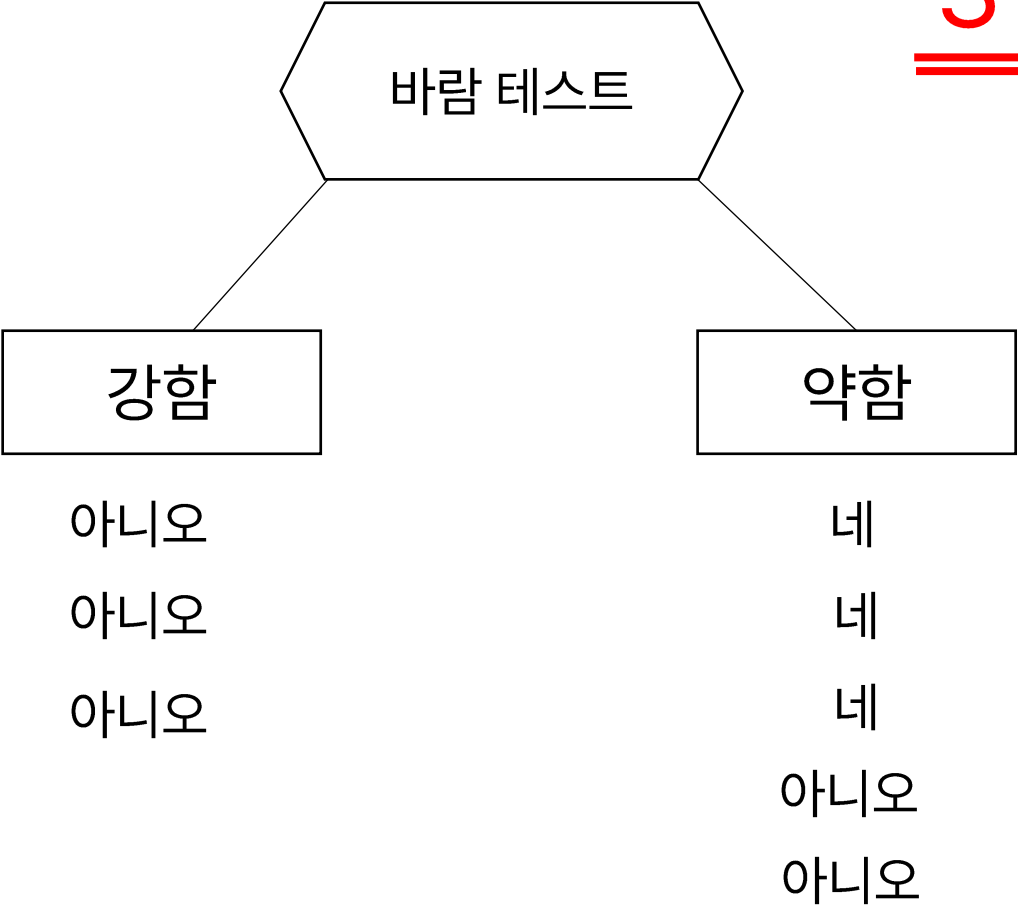
4점



배드민턴 플레이 여부

날씨	바람	온도	습도	배드민턴
맑음	강함	추움	보통	아니오
흐림	강함	더움	보통	아니오
맑음	약함	더움	보통	네
비	약함	보통	높음	네
맑음	약함	보통	낮음	네
흐림	약함	추움	높음	아니오
흐림	약함	보통	높음	아니오
맑음	강함	더움	낮음	아니오

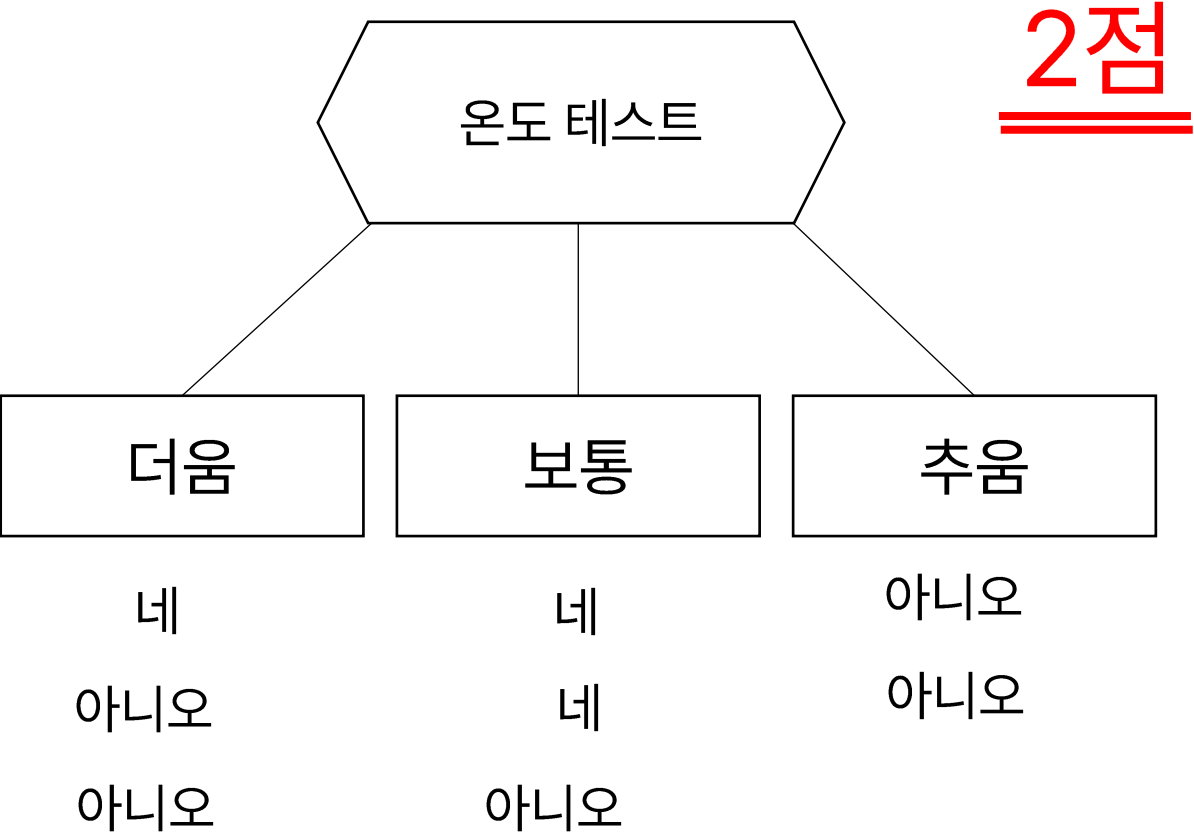
3점



배드민턴 플레이 여부

날씨	바람	온도	습도	배드민턴
맑음	강함	추움	보통	아니오
흐림	강함	더움	보통	아니오
맑음	약함	더움	보통	네
비	약함	보통	높음	네
맑음	약함	보통	낮음	네
흐림	약함	추움	높음	아니오
흐림	약함	보통	높음	아니오
맑음	강함	더움	낮음	아니오

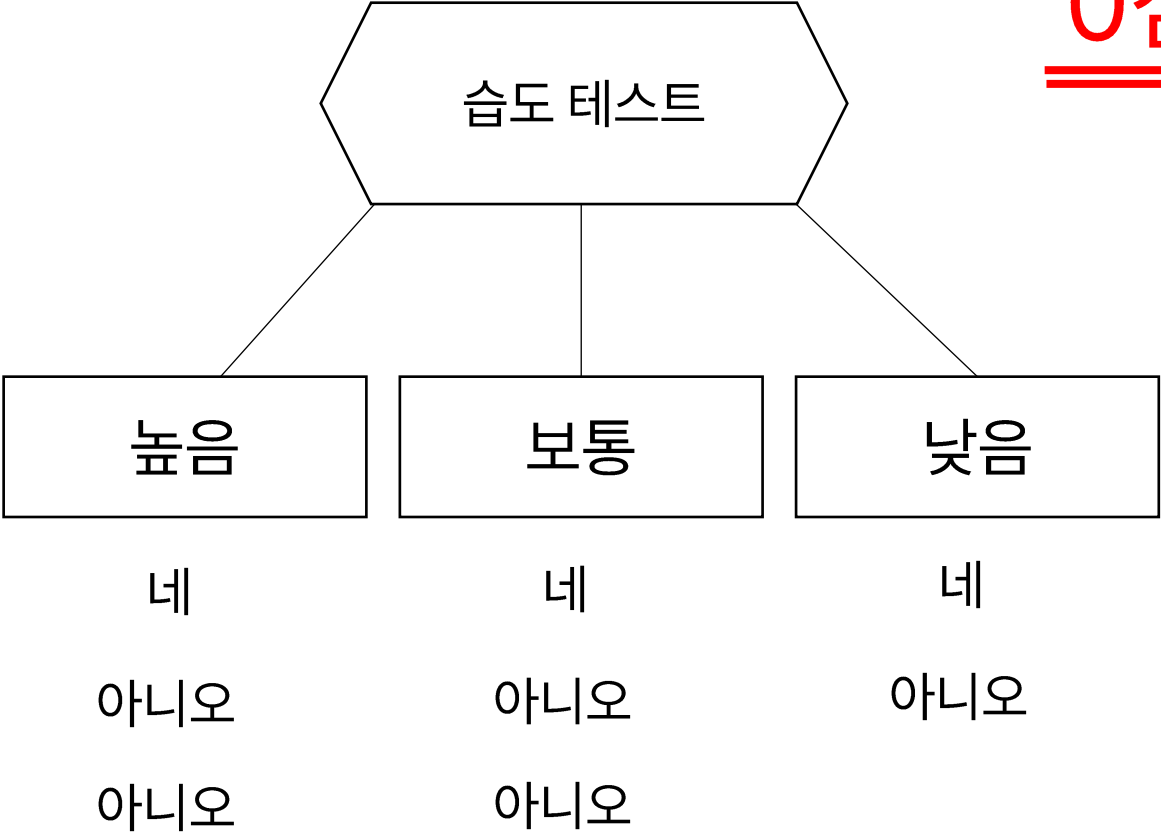
배드민턴 플레이 여부

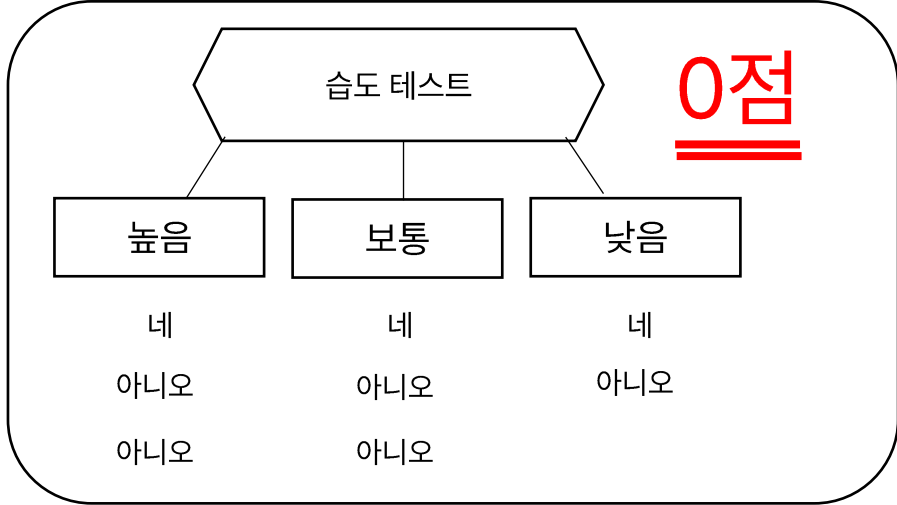
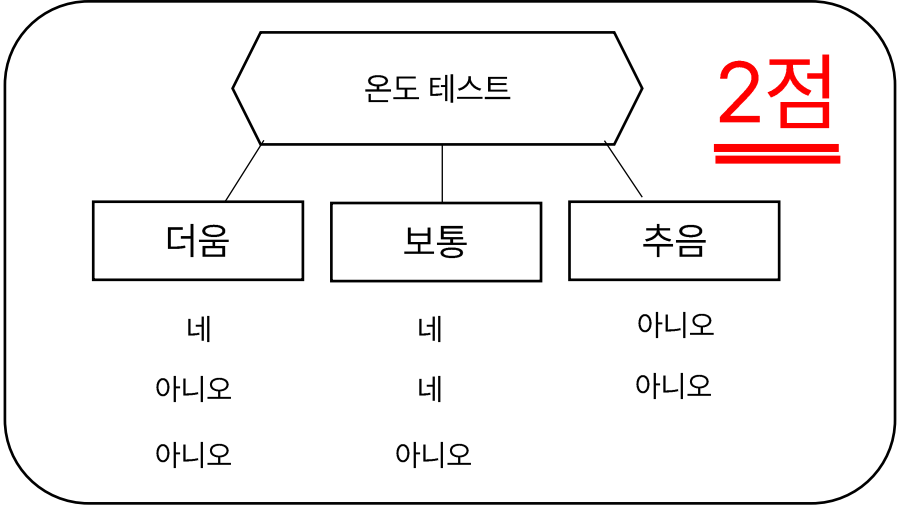
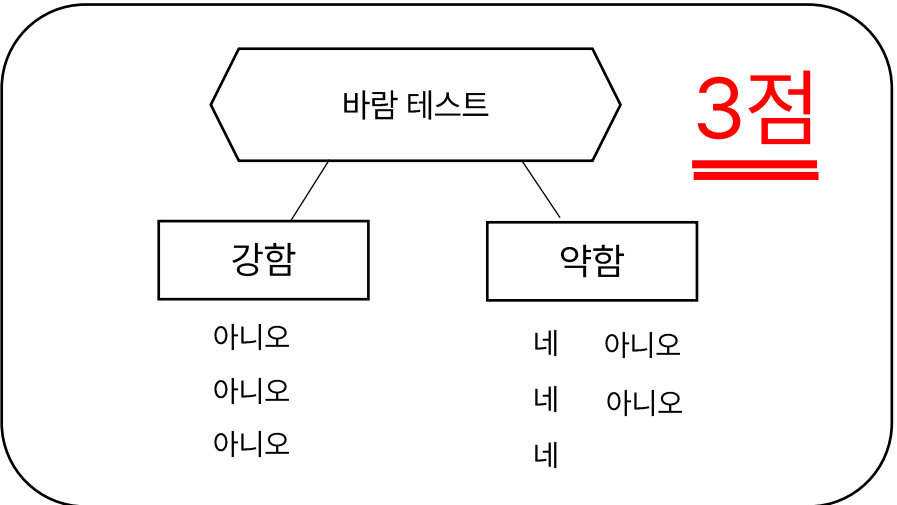
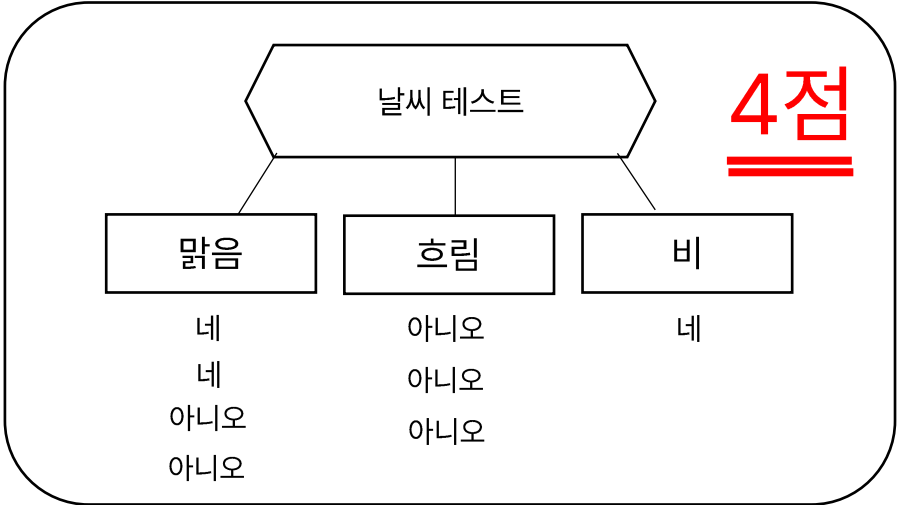


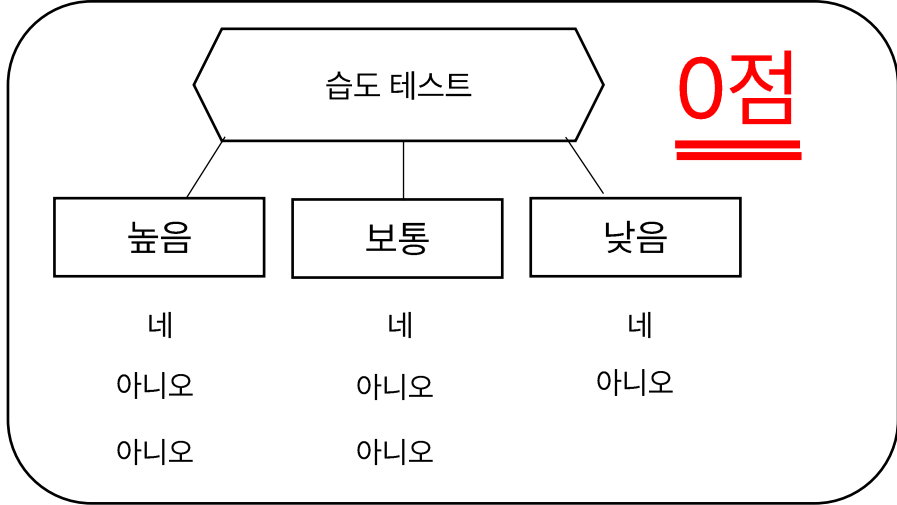
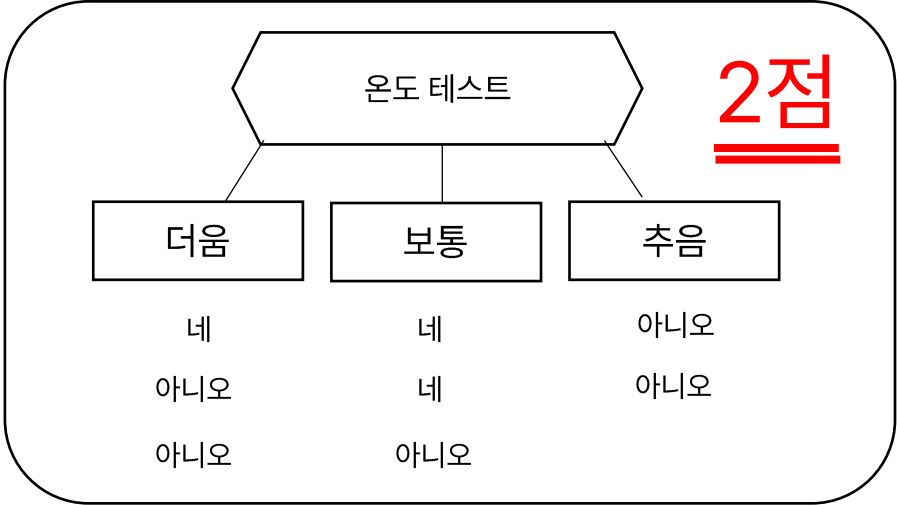
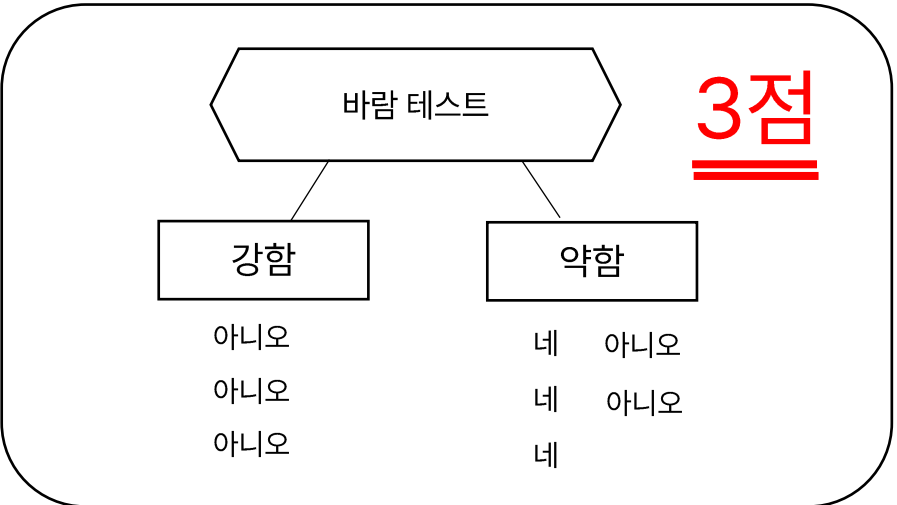
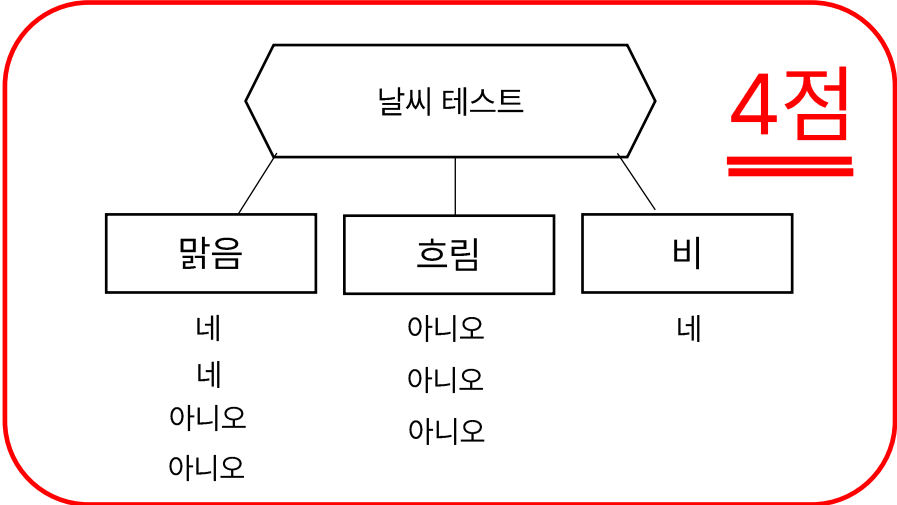
배드민턴 플레이 여부

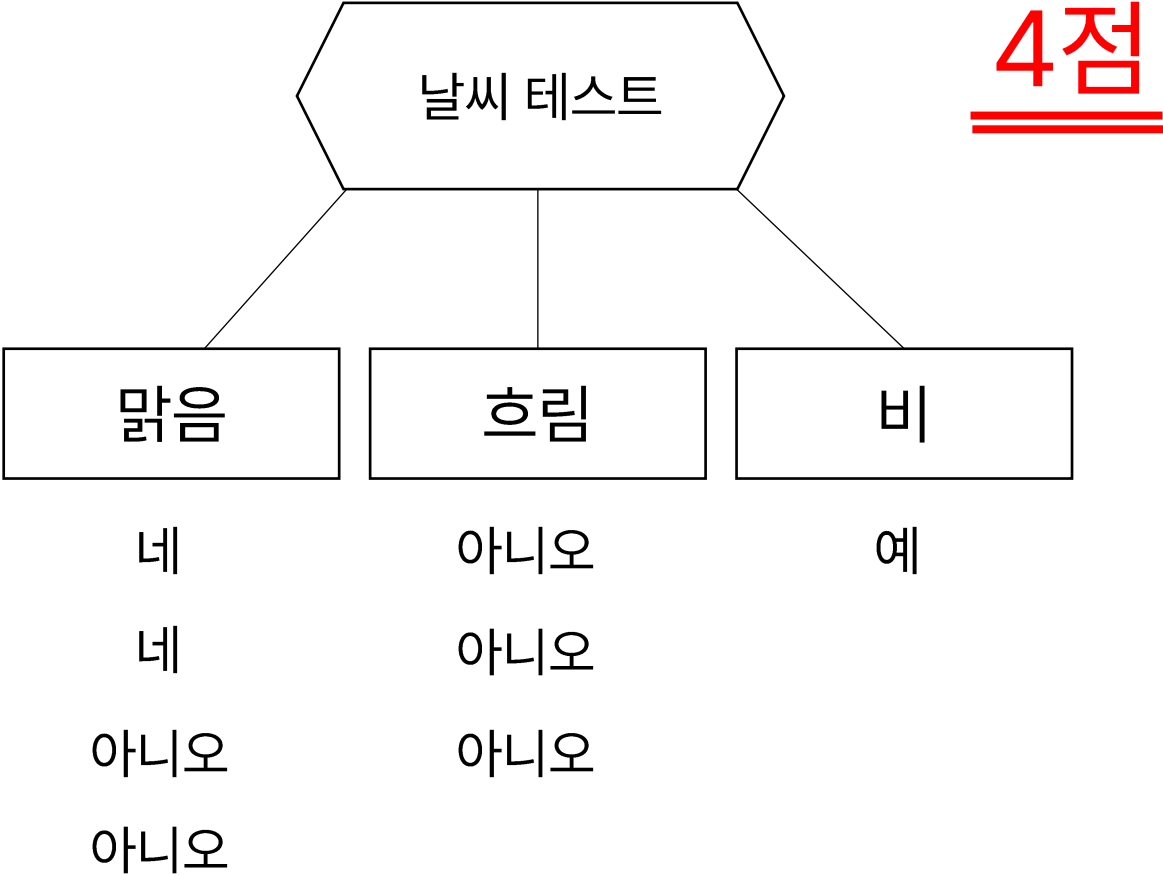
날씨	바람	온도	습도	배드민턴
맑음	강함	추움	보통	아니오
흐림	강함	더움	보통	아니오
맑음	약함	더움	보통	네
비	약함	보통	높음	네
맑음	약함	보통	낮음	네
흐림	약함	추움	높음	아니오
흐림	약함	보통	높음	아니오
맑음	강함	더움	낮음	아니오

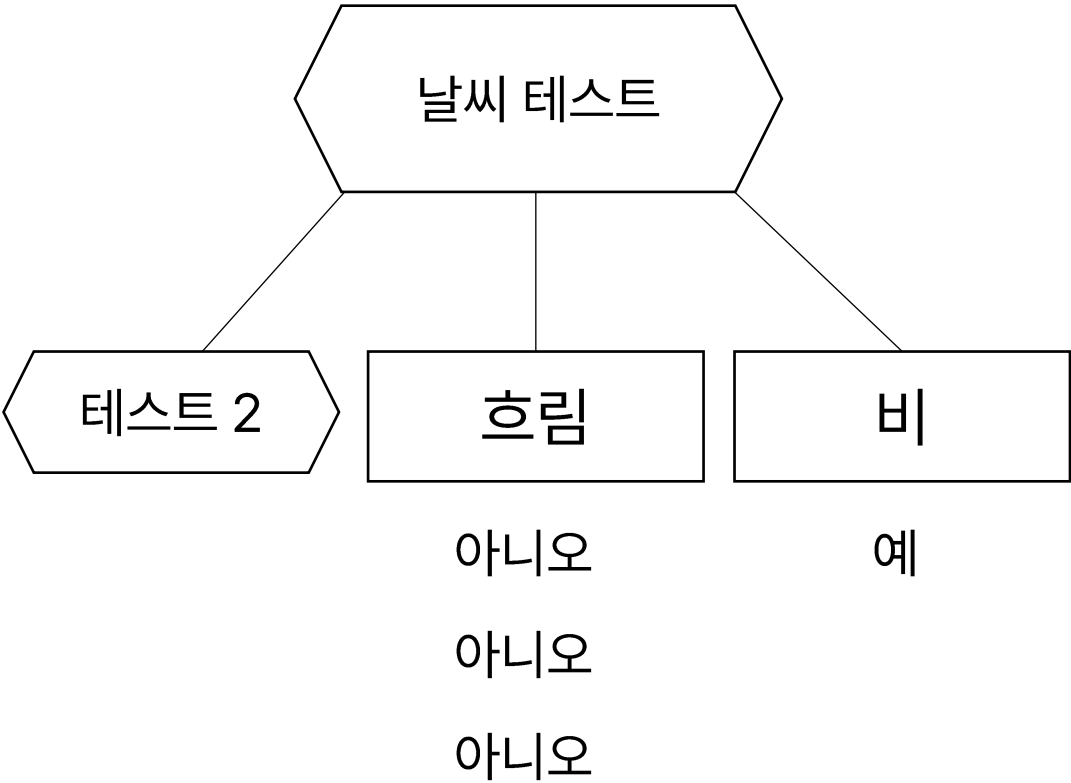
0점











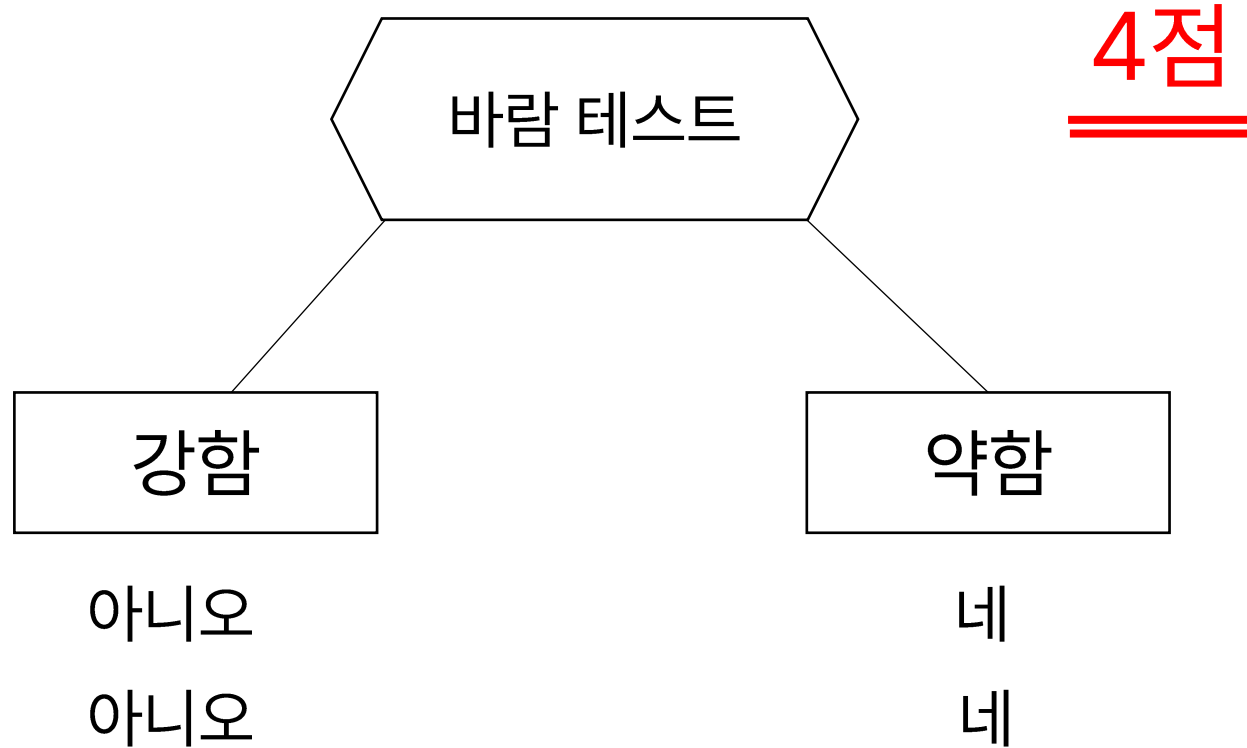
날씨 = 맑음 일때

날씨	바람	온도	습도	배드민턴
맑음	강함	추움	보통	아니오
흐림	강함	더움	보통	아니오
맑음	약함	더움	보통	네
비	약함	보통	높음	네
맑음	약함	보통	낮음	네
흐림	약함	추움	높음	아니오
흐림	약함	보통	높음	아니오
맑음	강함	더움	낮음	아니오

날씨 = 맑음 일때

날씨	바람	온도	습도	배드민턴
맑음	강함	추움	보통	아니오
흐림	강함	더움	보통	아니오
맑음	약함	더움	보통	네
비	약함	보통	높음	네
맑음	약함	보통	낮음	네
흐림	약함	추움	높음	아니오
흐림	약함	보통	높음	아니오
맑음	강함	더움	낮음	아니오

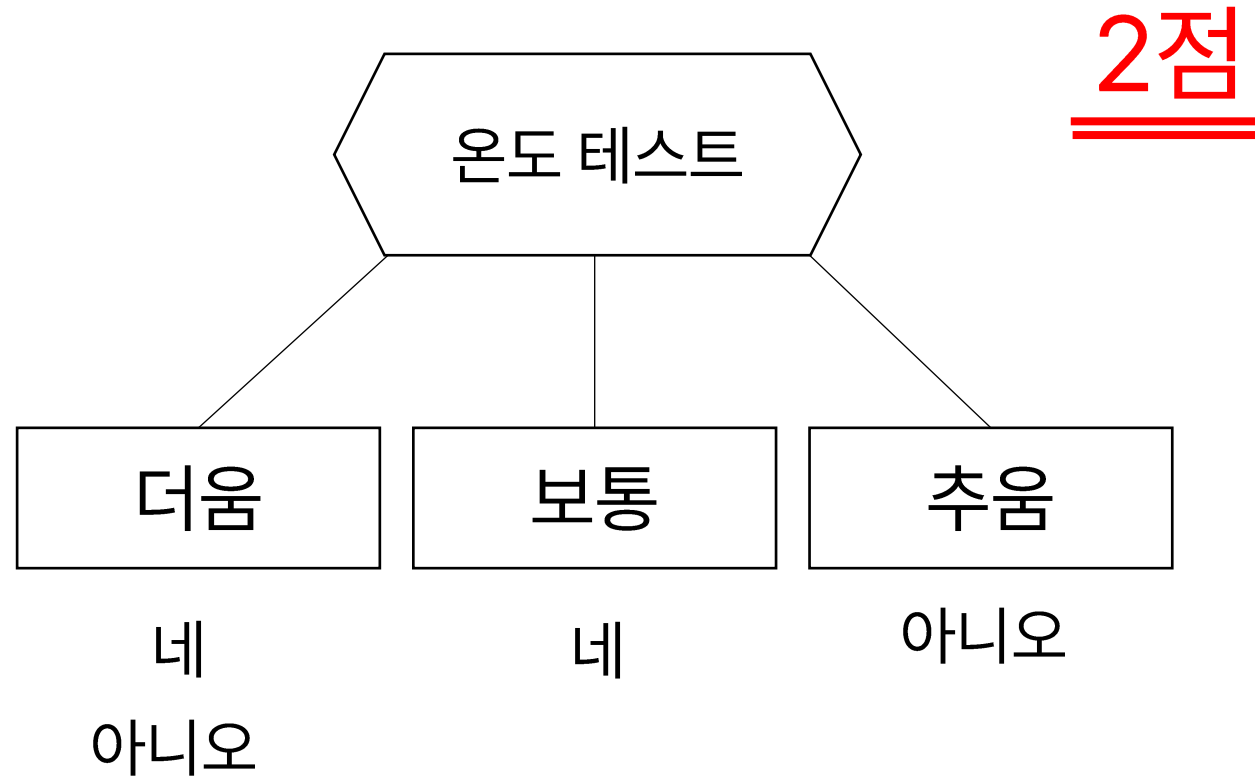
날씨 = 맑음 기준으로 바람 테스트



날씨 = 맑음 일때

날씨	바람	온도	습도	배드민턴
맑음	강함	추움	보통	아니오
흐림	강함	더움	보통	아니오
맑음	약함	더움	보통	네
비	약함	보통	높음	네
맑음	약함	보통	낮음	네
흐림	약함	추움	높음	아니오
흐림	약함	보통	높음	아니오
맑음	강함	더움	낮음	아니오

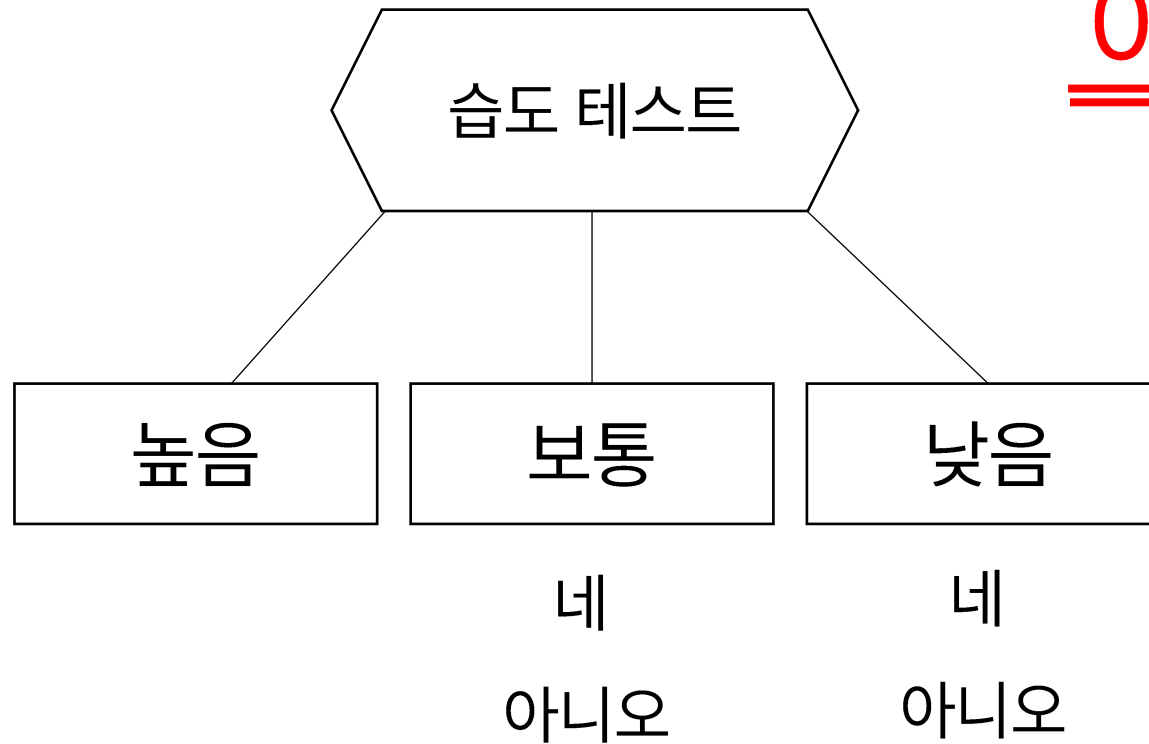
날씨 = 맑음 기준으로 온도 테스트



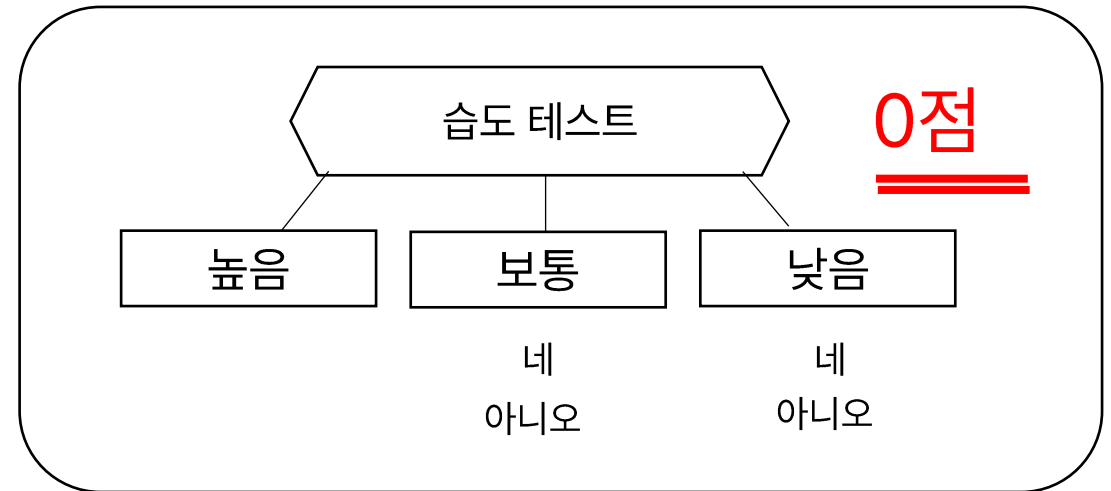
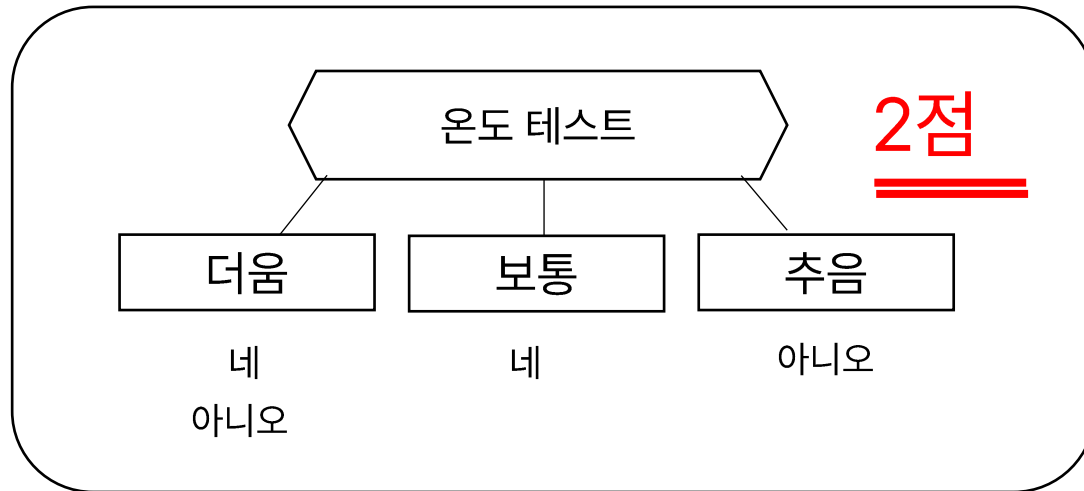
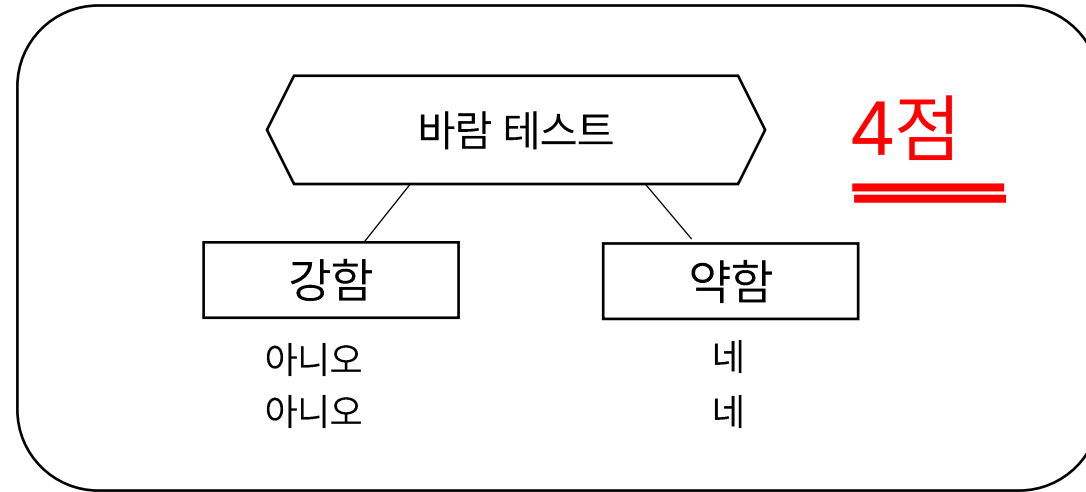
날씨 = 맑음 일때

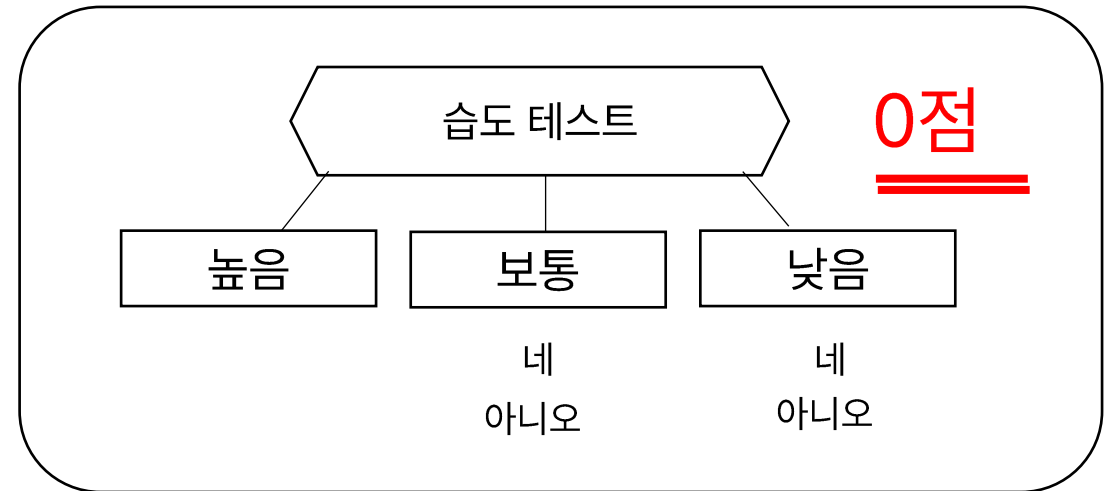
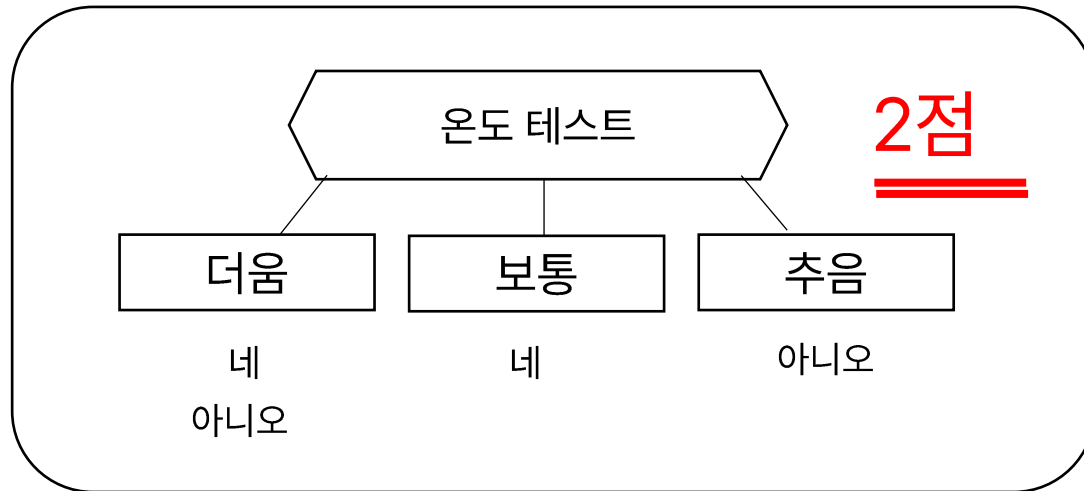
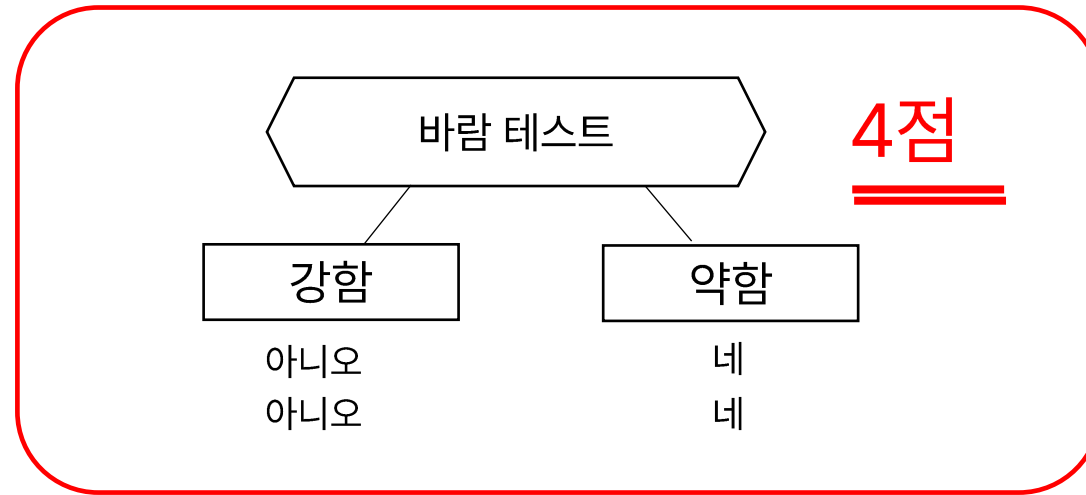
날씨	바람	온도	습도	배드민턴
맑음	강함	추움	보통	아니오
흐림	강함	더움	보통	아니오
맑음	약함	더움	보통	네
비	약함	보통	높음	네
맑음	약함	보통	낮음	네
흐림	약함	추움	높음	아니오
흐림	약함	보통	높음	아니오
맑음	강함	더움	낮음	아니오

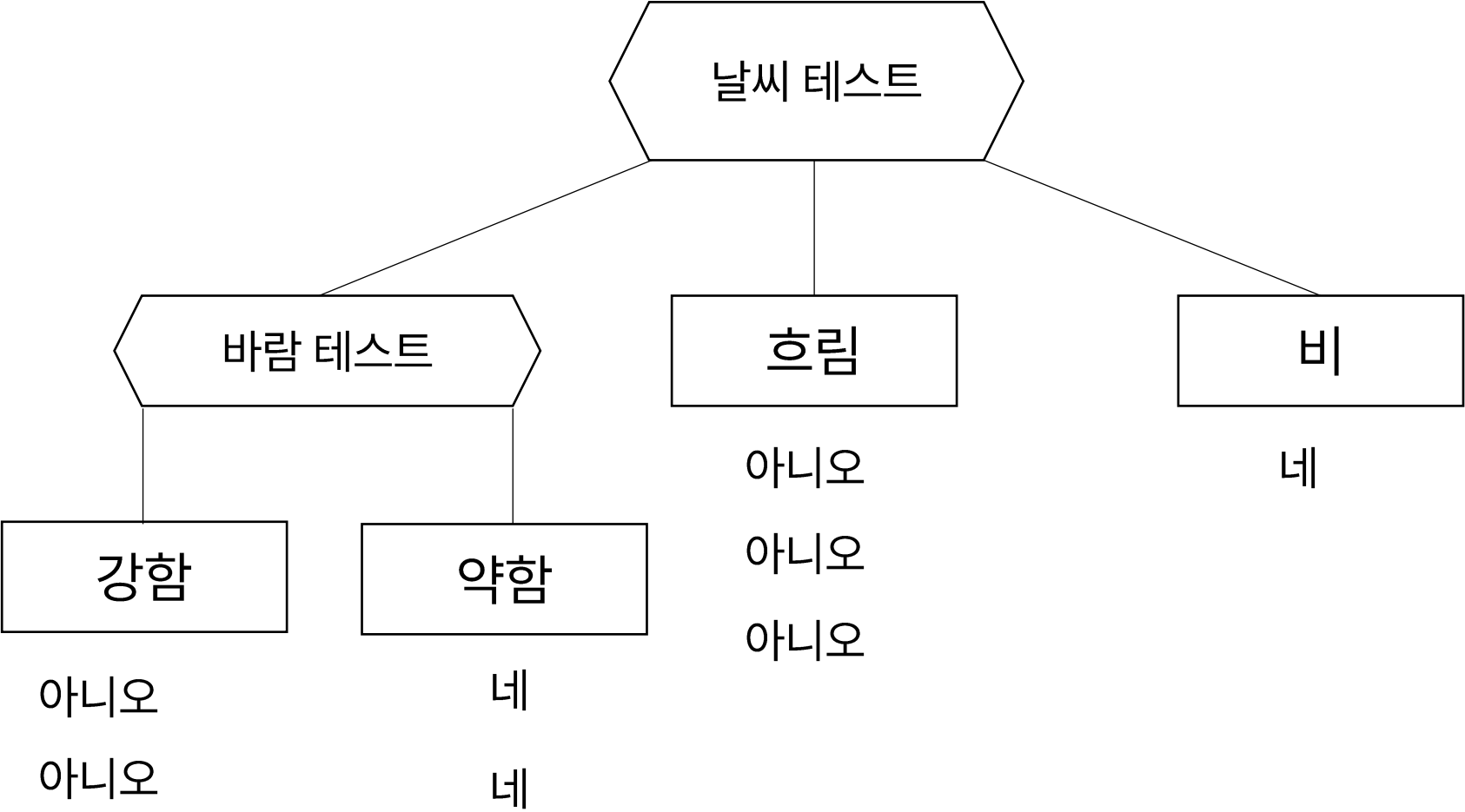
날씨 = 맑음 기준으로 습도 테스트



0점







실제 스코어링 방법

- ✓ 노드 별 엔트로피 측정
- ✓ 엔트로피란 무질서 정도를 나타내는 척도
- ✓ 엔트로피는 낮을 수록 좋은
- ✓ 노드별 엔트로피 측정 후 테스트 전체 퀄리티 측정

실제 스코어링 방법

1.10.7.1. Classification criteria

If a target is a classification outcome taking on values $0, 1, \dots, K-1$, for node m , let

$$p_{mk} = \frac{1}{n_m} \sum_{y \in Q_m} I(y = k)$$

be the proportion of class k observations in node m . If m is a terminal node, `predict_proba` for this region is set to p_{mk} . Common measures of impurity are the following.

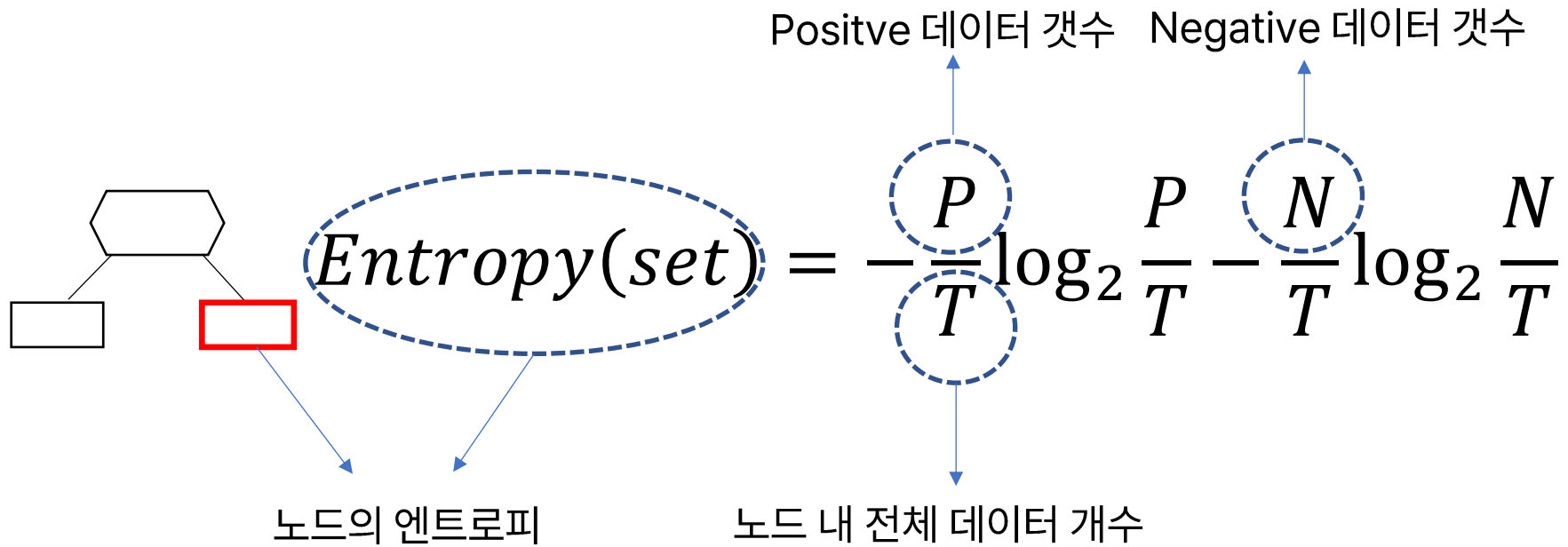
Gini:

$$H(Q_m) = \sum_k p_{mk}(1 - p_{mk})$$

Log Loss or Entropy:

$$H(Q_m) = - \sum_k p_{mk} \log(p_{mk})$$

엔트로피(Entropy)



The diagram illustrates the concept of Entropy in a decision tree. On the left, a decision tree node is shown as a hexagon with two child nodes, represented by rectangles. The right child node is highlighted with a red border. A blue dashed oval labeled $Entropy(set)$ is positioned to the right of the node. Two blue arrows point from this oval to the text '노드의 엔트로피' (Node Entropy) below. To the right of the oval is the formula for entropy:
$$Entropy(set) = -\frac{P}{T} \log_2 \frac{P}{T} - \frac{N}{T} \log_2 \frac{N}{T}$$
 The terms $\frac{P}{T}$ and $\frac{N}{T}$ in the formula are each enclosed in a blue dashed circle. A blue arrow points from the first circle to the text 'Positive 데이터 갯수' (Positive data count) above. Another blue arrow points from the second circle to the text 'Negative 데이터 갯수' (Negative data count) above. A third blue arrow points from the denominator T in the first term to the text '노드 내 전체 데이터 개수' (Total data count in node) below.

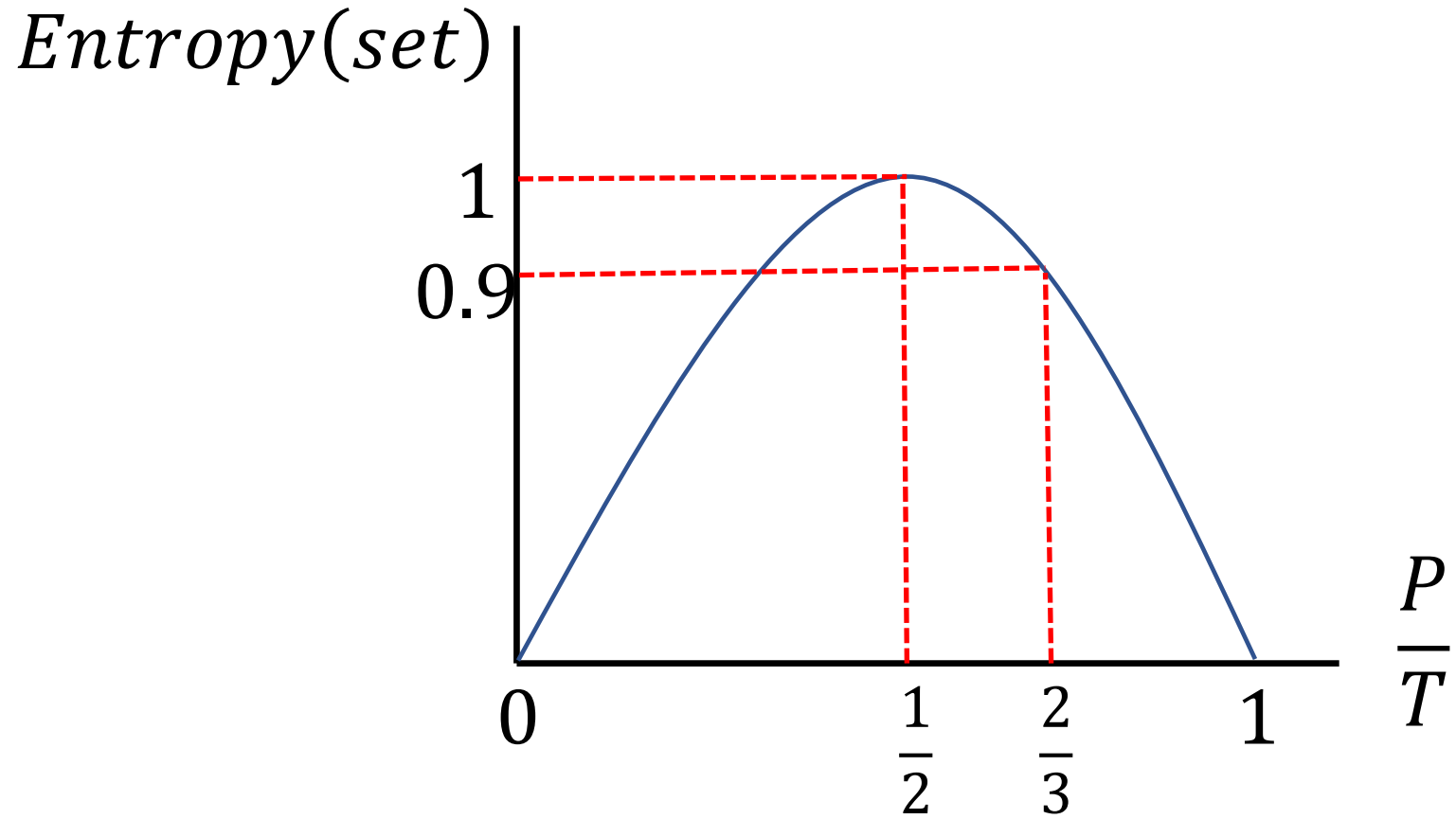
Positive 데이터 갯수 Negative 데이터 갯수

$Entropy(set) = -\frac{P}{T} \log_2 \frac{P}{T} - \frac{N}{T} \log_2 \frac{N}{T}$

노드의 엔트로피 노드 내 전체 데이터 개수

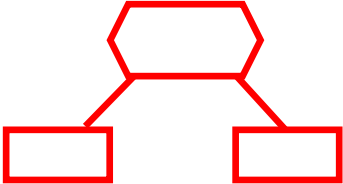
엔트로피는 낮을 수록 좋음!

엔트로피(Entropy)

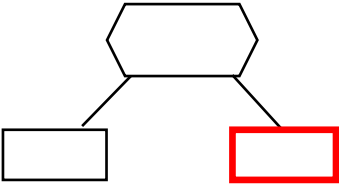


테스트 스코어

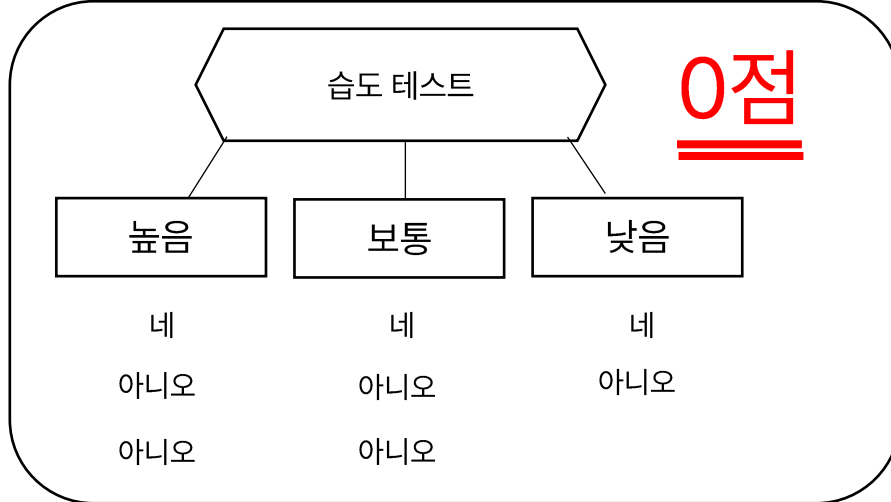
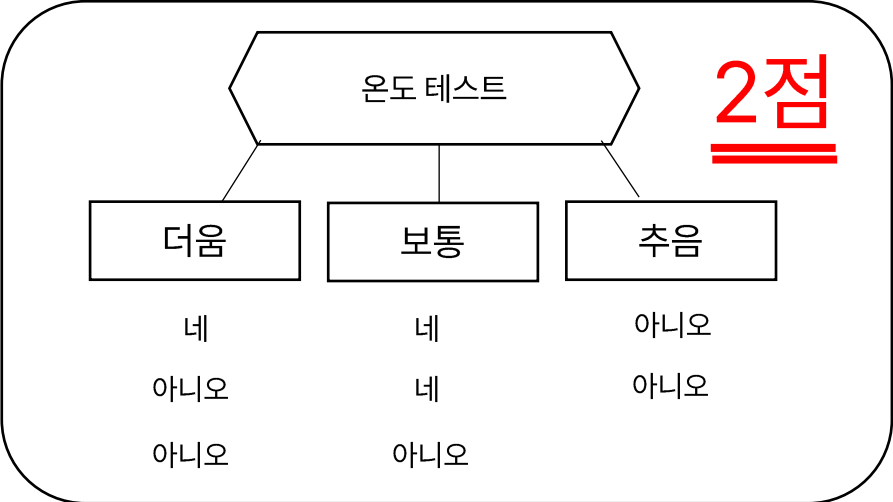
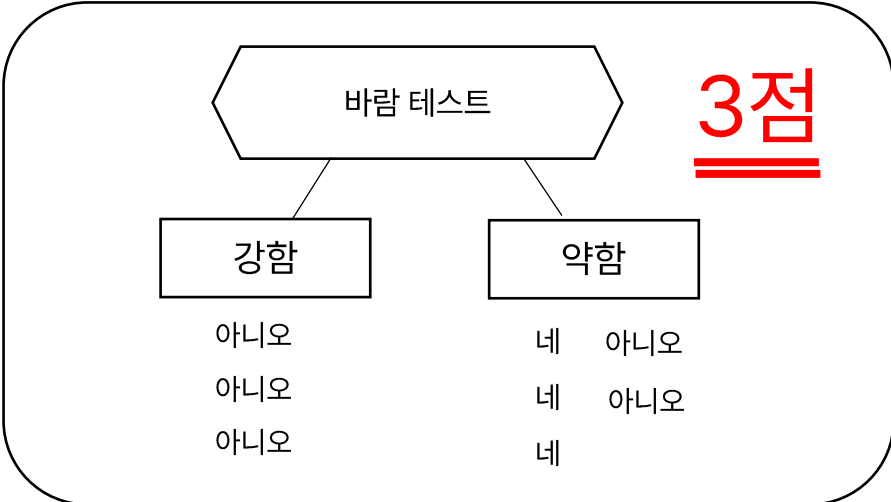
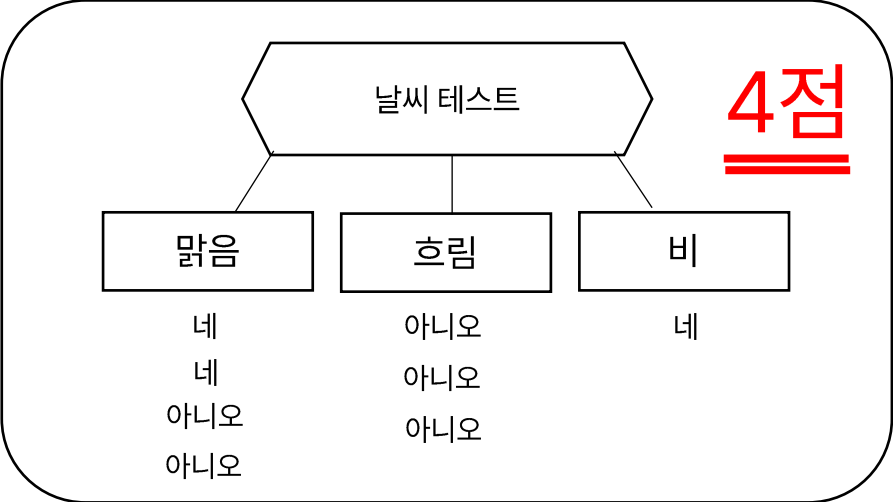
테스트 전체



각 노드의 엔트로피


$$Q(Test) = \sum_{\substack{\text{sets} \\ \text{produced}}} D(sets) \times \frac{\text{해당 노드 데이터 수}}{\text{테스트 전체 데이터 수}}$$

노드 개수



$$D(set) = -\frac{P}{T} \log_2 \frac{P}{T} - \frac{N}{T} \log_2 \frac{N}{T}$$

$$T = 4$$

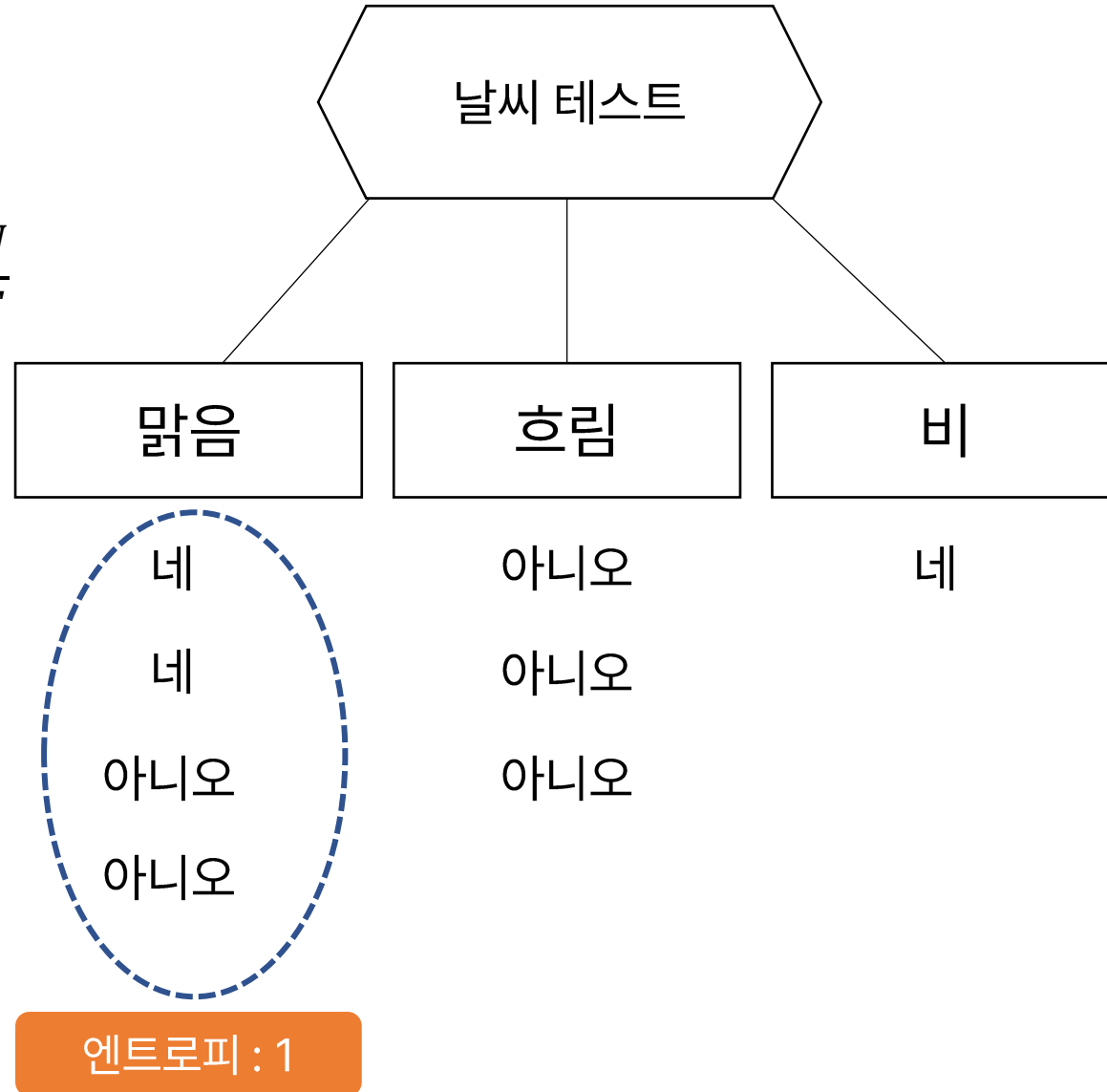
$$P = 2$$

$$N = 2$$

$$D(set) = -\frac{2}{4} \log_2 \frac{2}{4} - \frac{2}{4} \log_2 \frac{2}{4}$$

$$= \frac{1}{2} + \frac{1}{2}$$

$$= 1$$



Section
의사결정나무개념

$$D(set) = -\frac{P}{T} \log_2 \frac{P}{T} - \frac{N}{T} \log_2 \frac{N}{T}$$

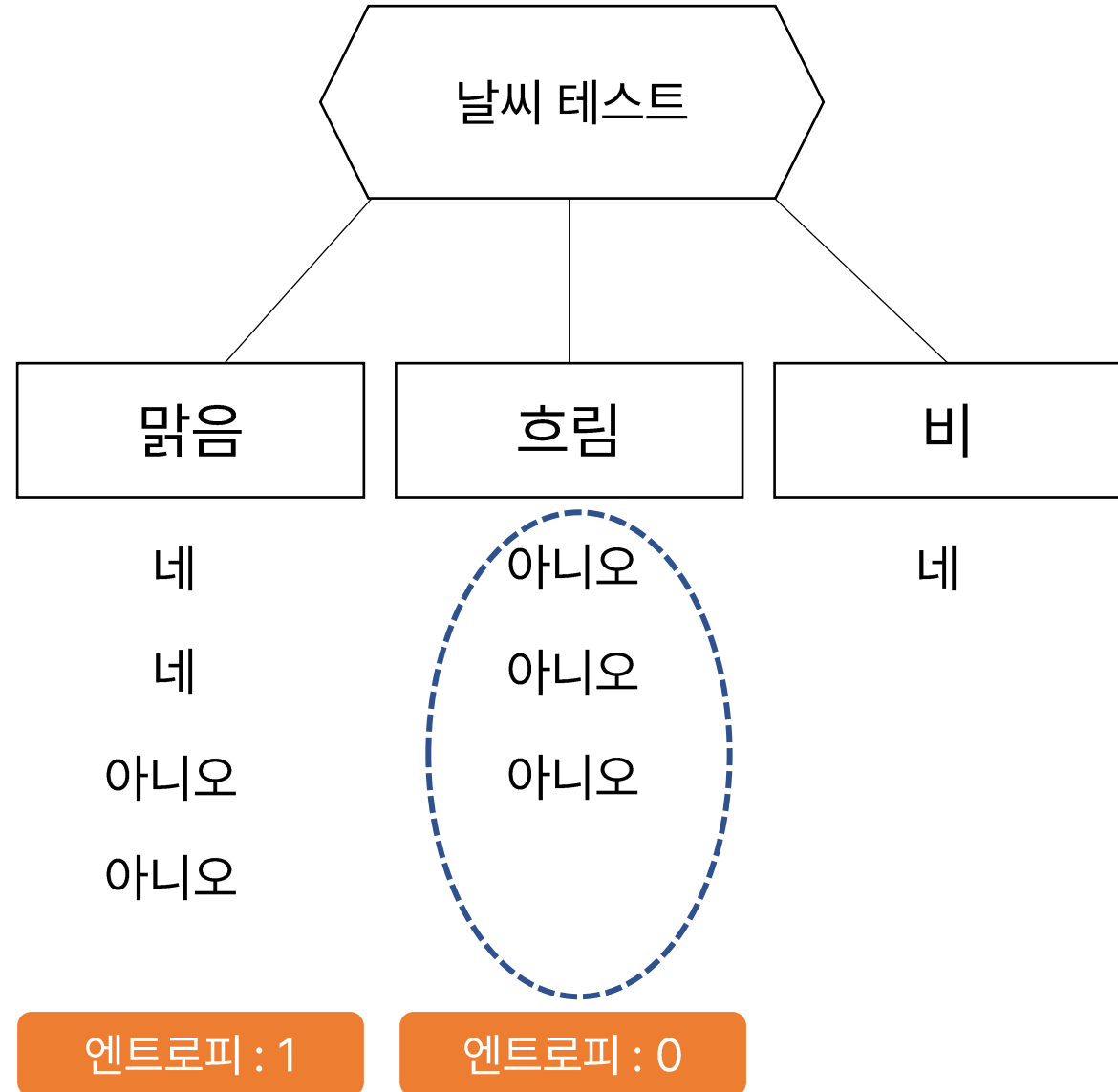
$$T = 3$$

$$P = 0$$

$$N = 3$$

$$D(set) = -\frac{0}{3} \log_2 \frac{0}{3} - \frac{3}{3} \log_2 \frac{3}{3}$$

$$= 0$$

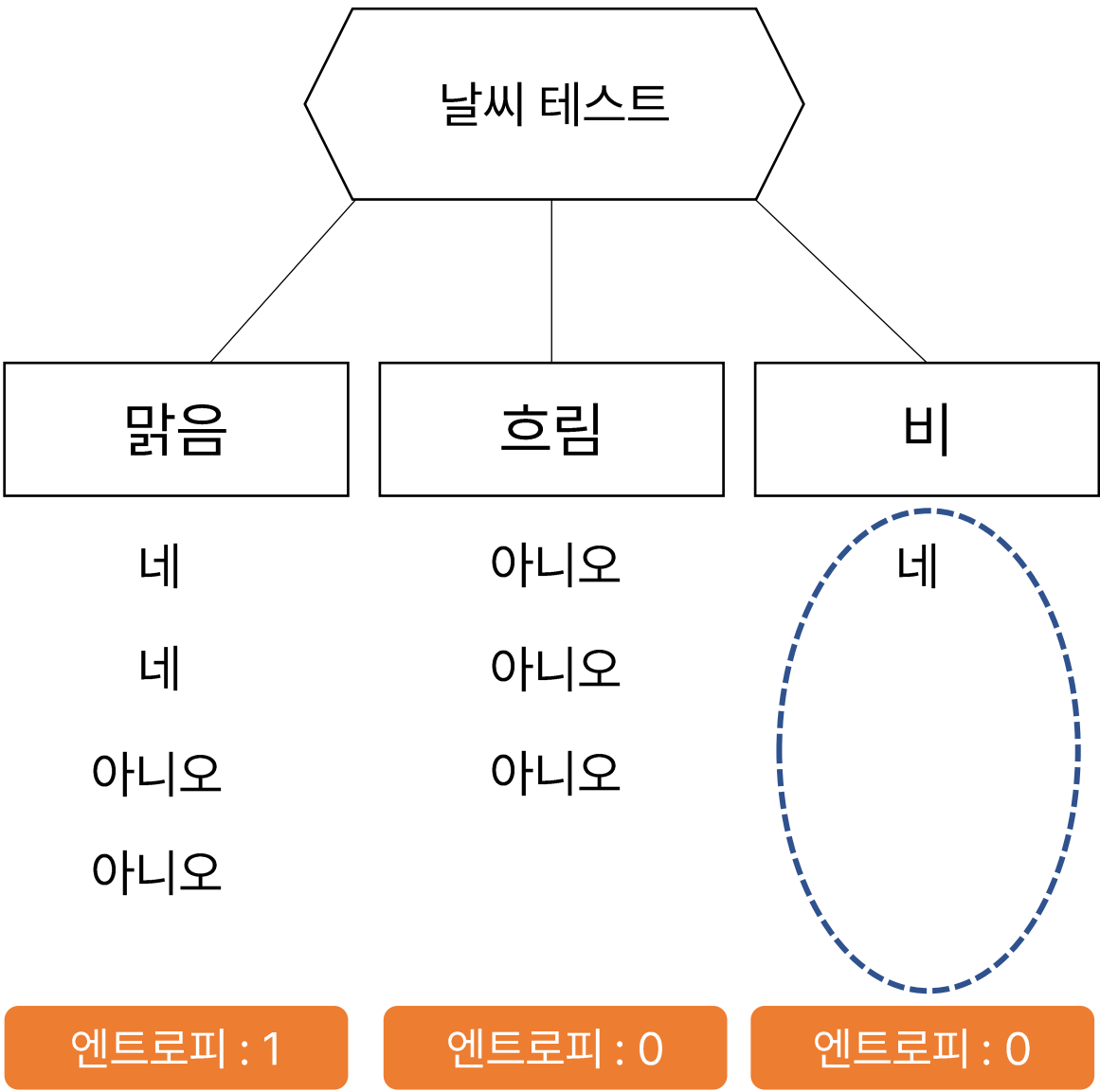


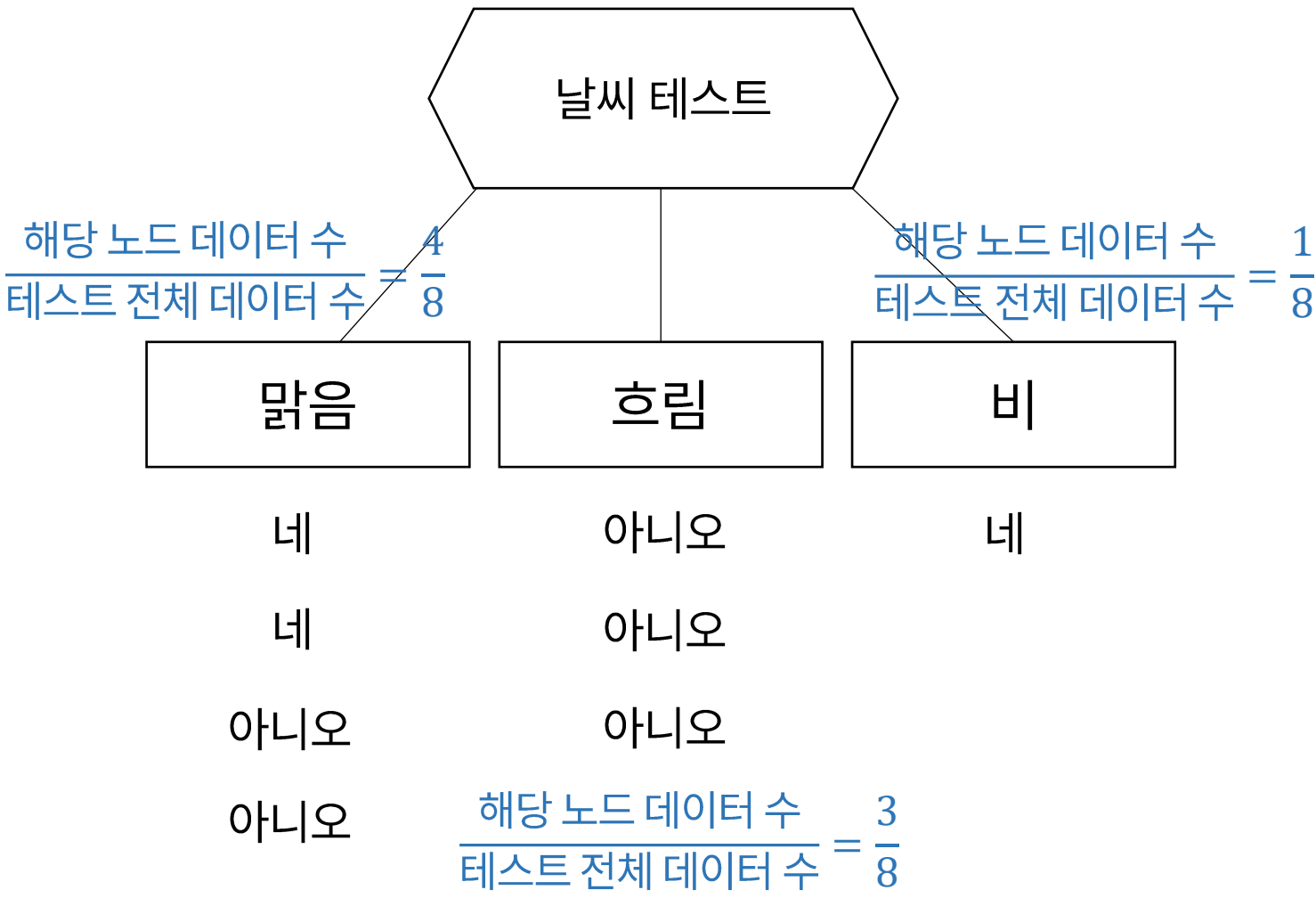
$$D(set) = -\frac{P}{T} \log_2 \frac{P}{T} - \frac{N}{T} \log_2 \frac{N}{T}$$

T = 1
P = 1
N = 0

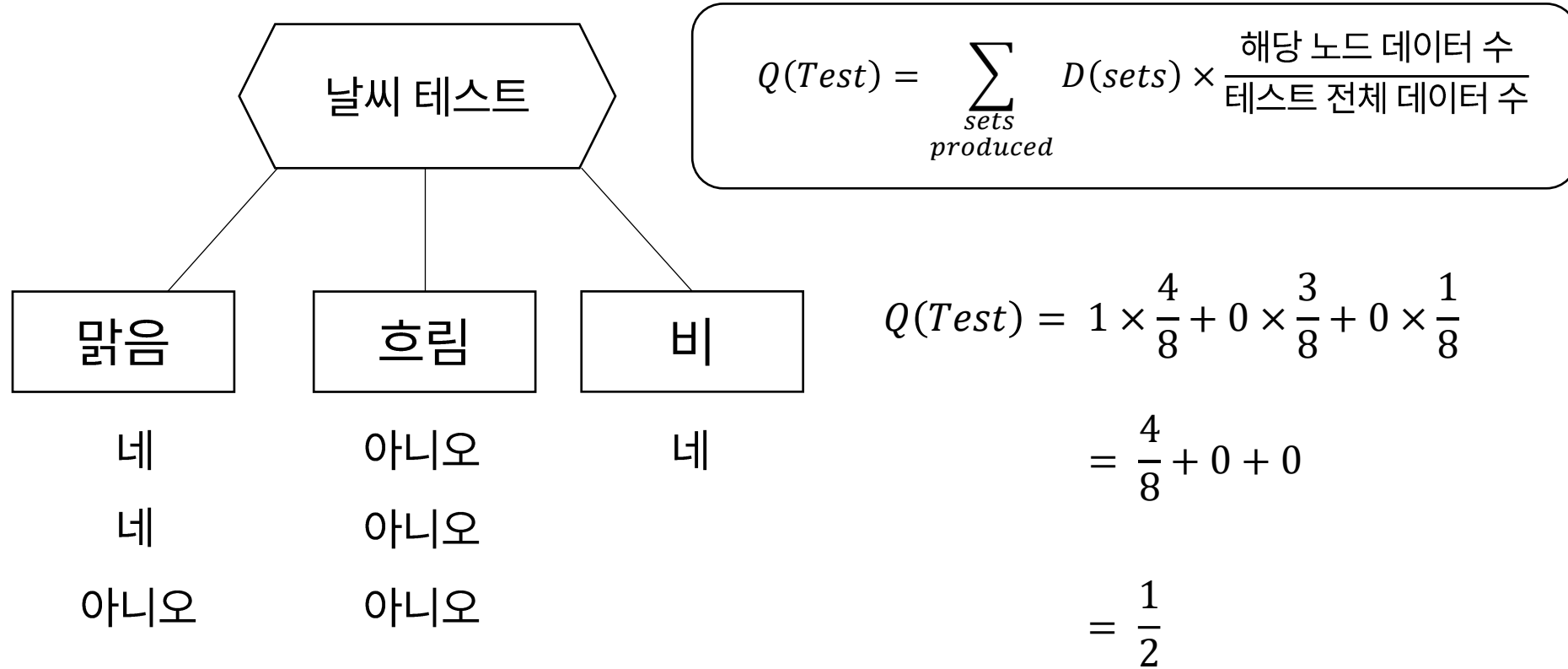
$$D(set) = -\frac{1}{1} \log_2 \frac{1}{1} - \frac{0}{1} \log_2 \frac{0}{1}$$

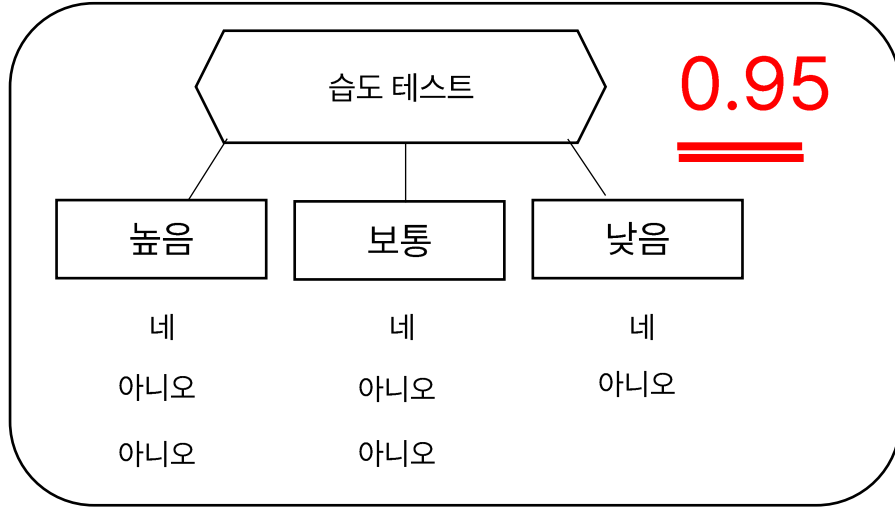
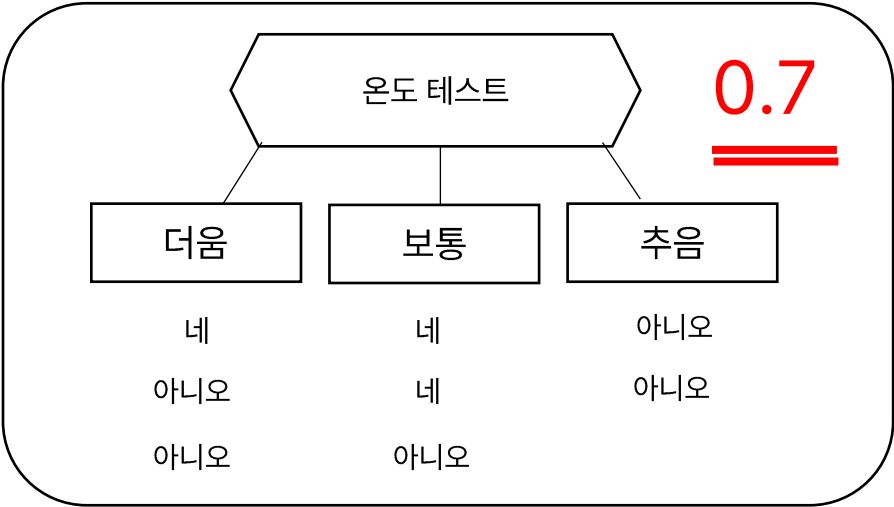
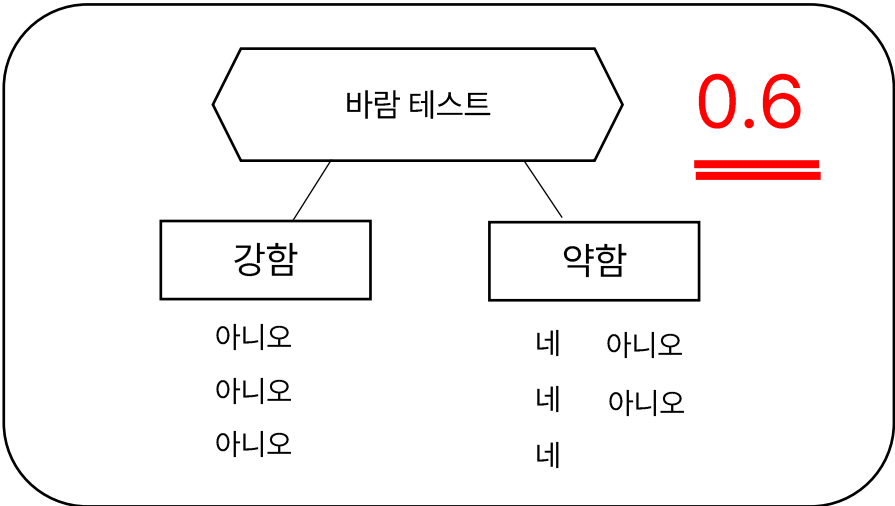
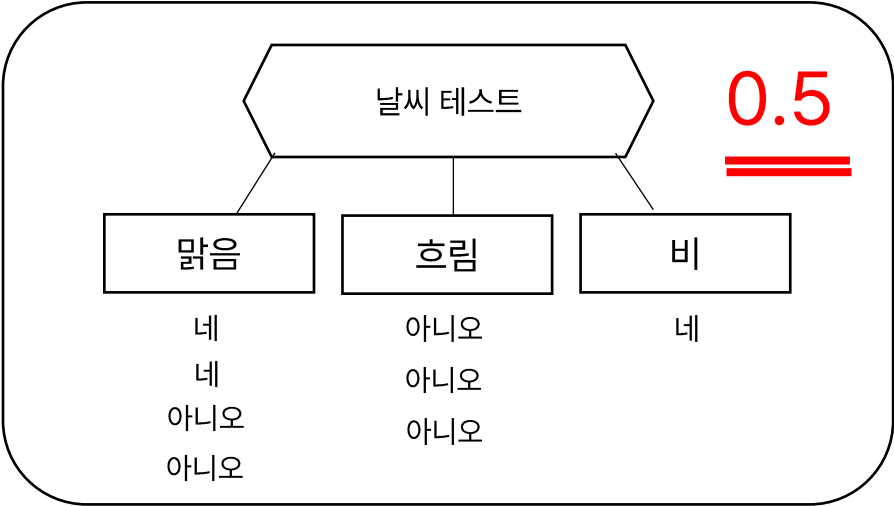
= 0

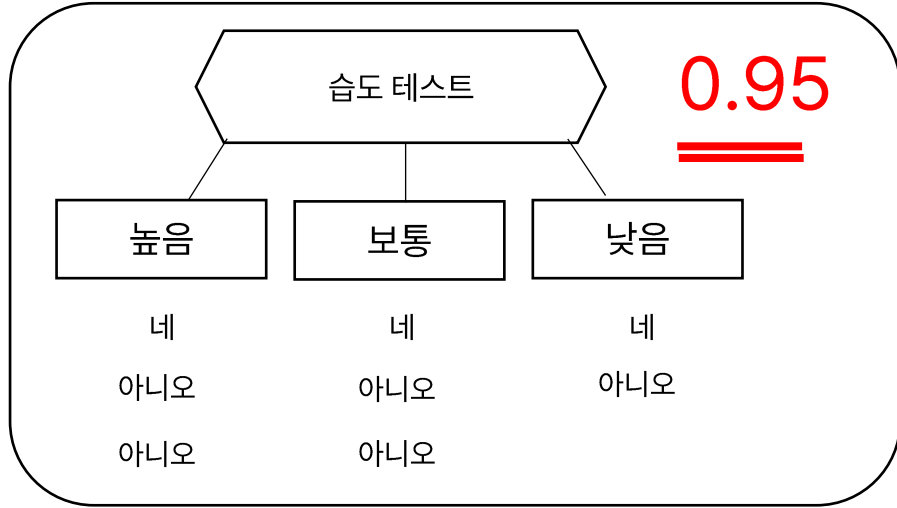
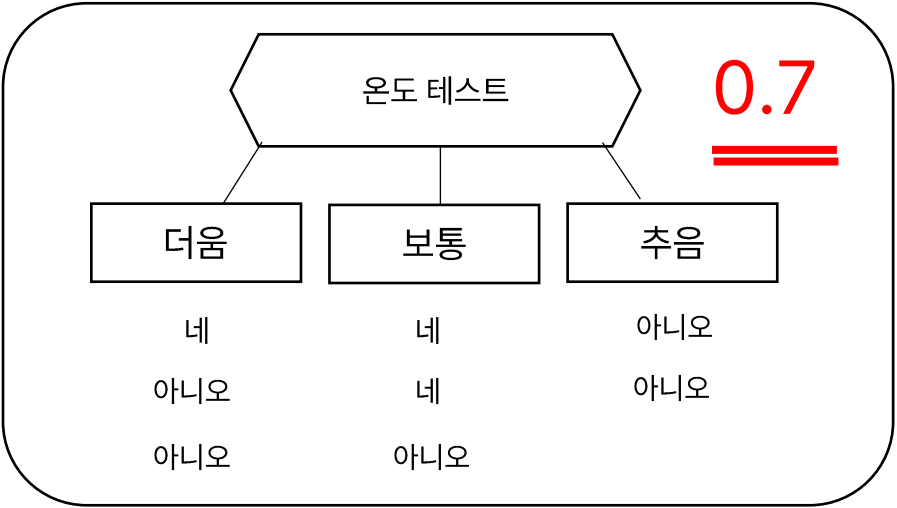
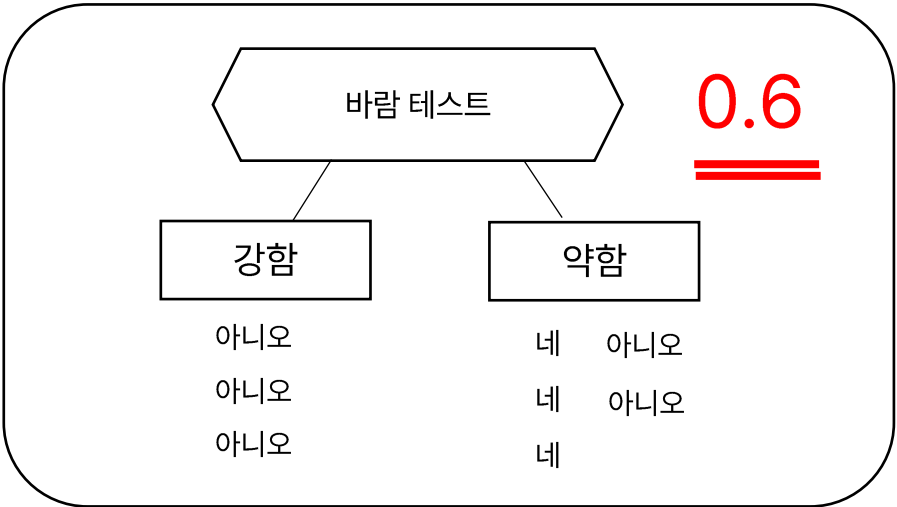
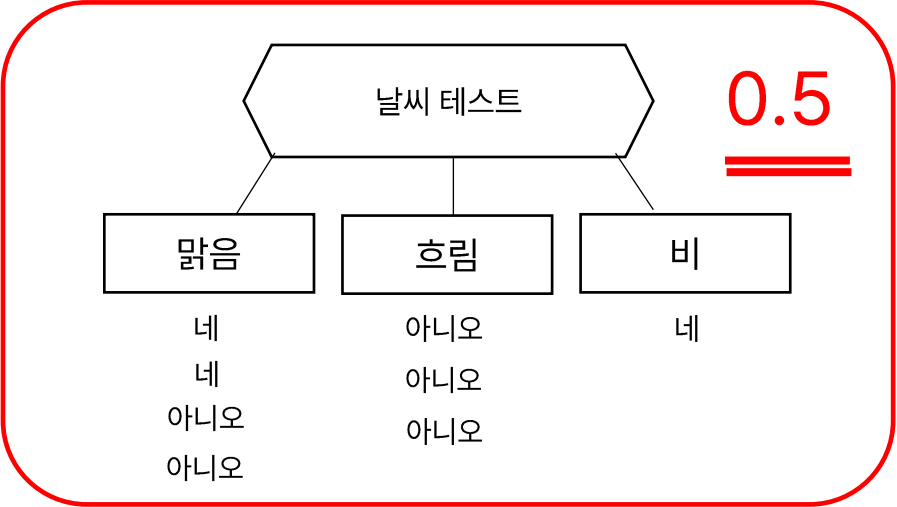




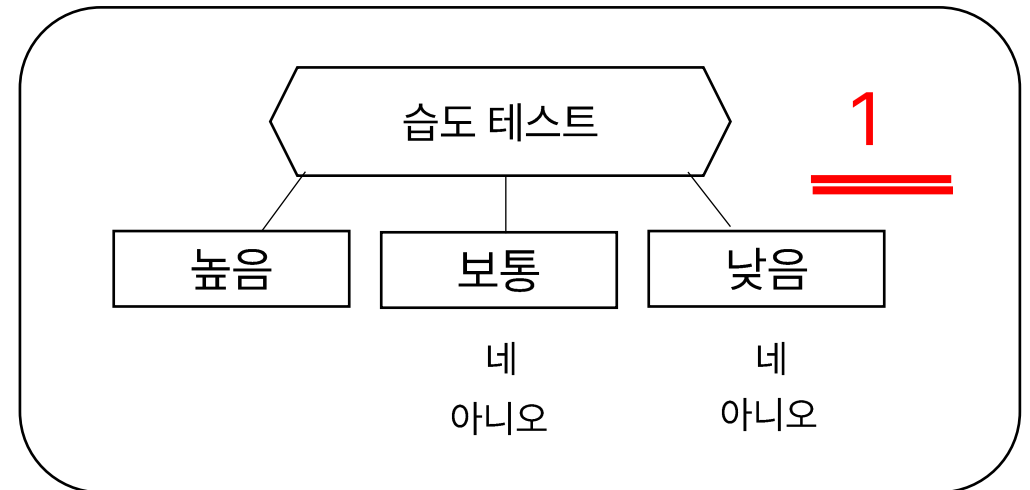
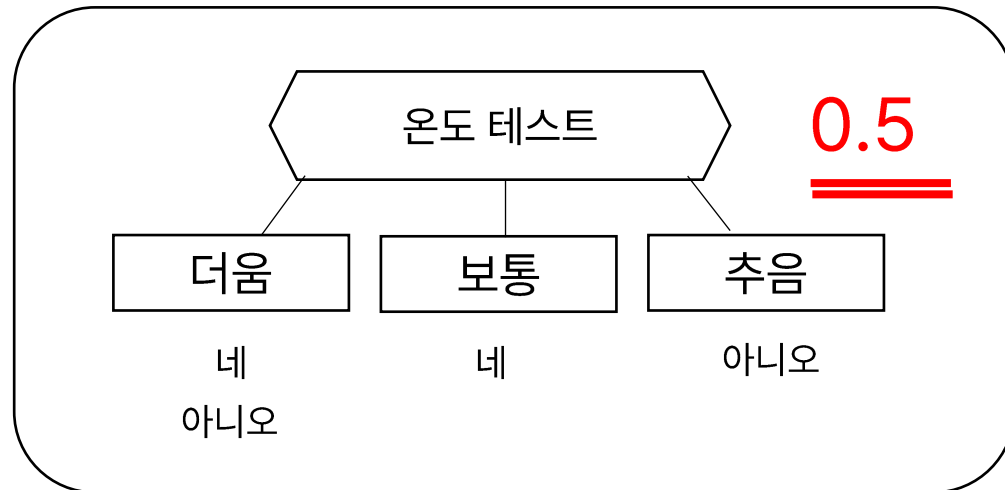
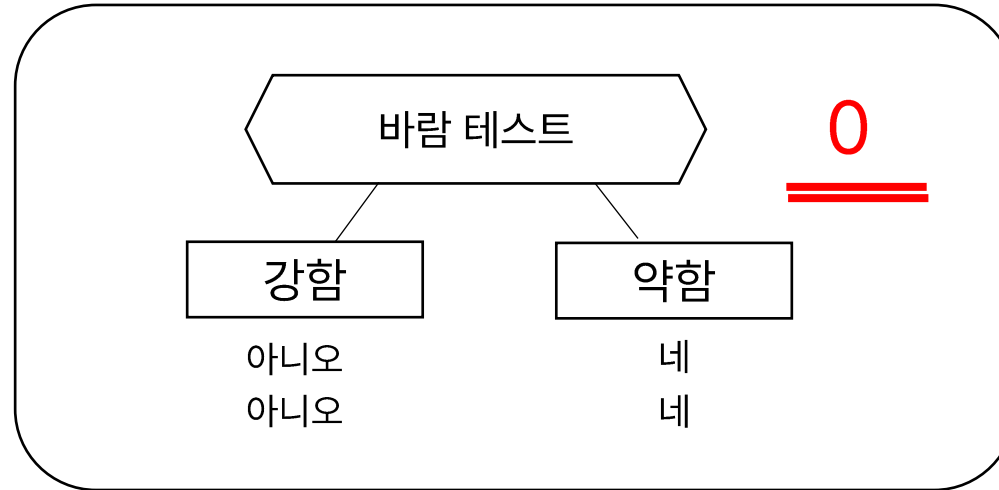
Section
의사결정나무개념

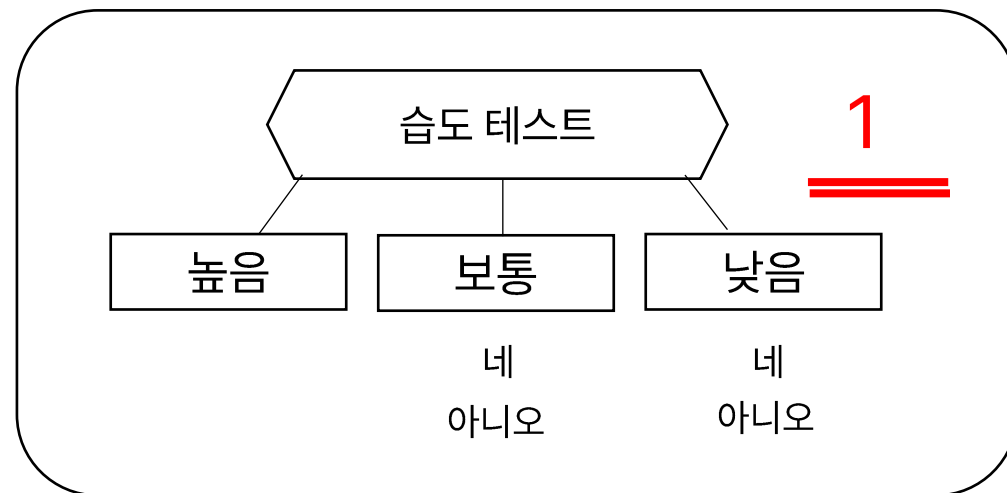
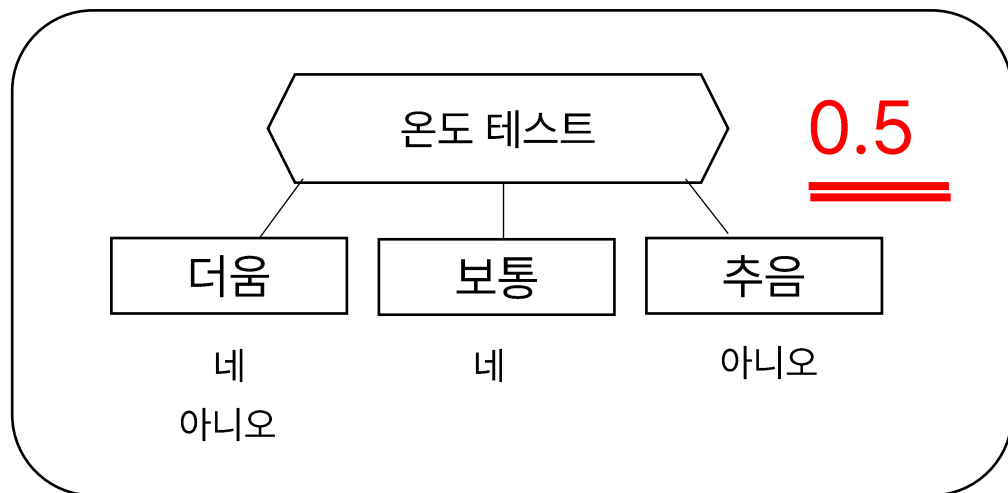
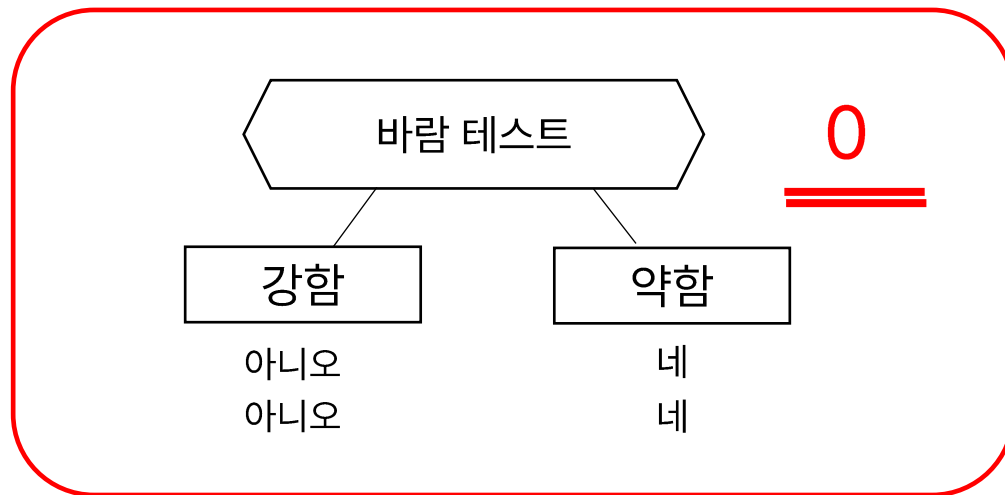






Section
의사결정나무개념

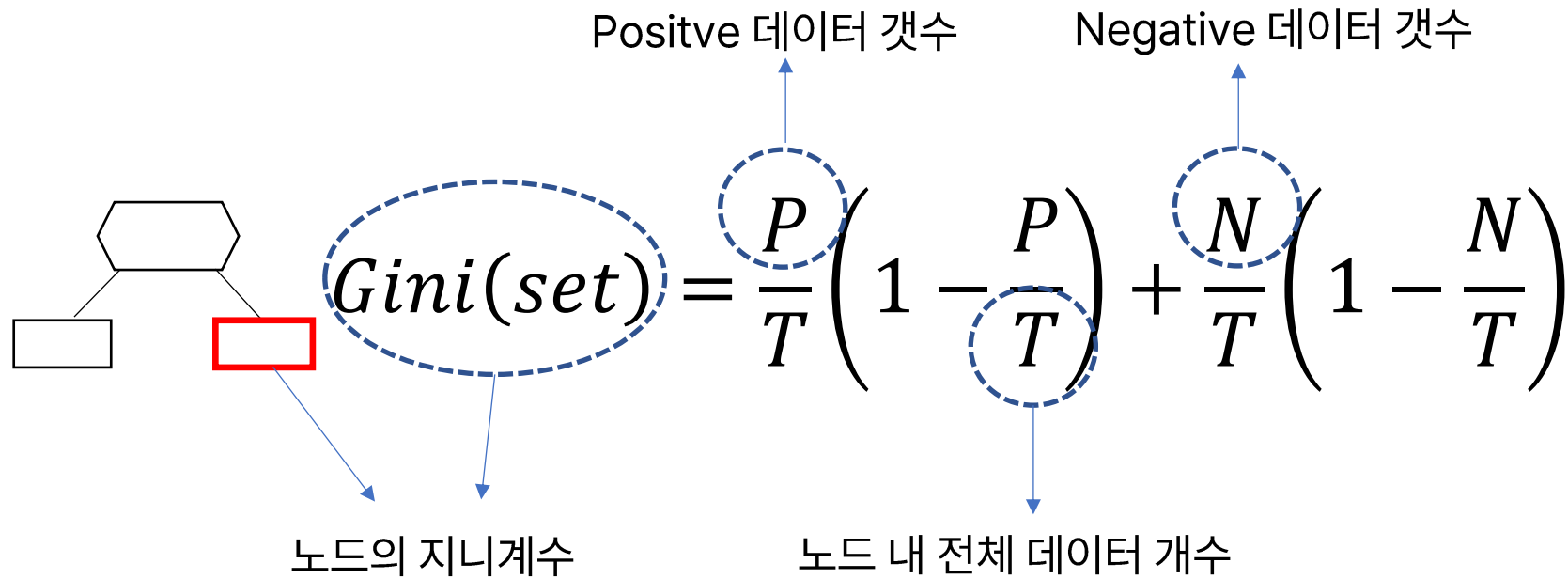




실제 스코어링 방법

- ✓ 노드 별 지니계수(Gini index) 측정
- ✓ 지니계수는 낮을 수록 좋음
- ✓ 노드별 지니계수 측정 후 테스트 전체 퀄리티 측정

지니계수(Gini index)



The diagram illustrates the Gini index calculation for a node in a decision tree. On the left, a decision tree structure is shown with a root node (hexagon) and two child nodes (rectangles). The right child node is highlighted with a red border. An arrow points from this node to the text '노드의 지니계수' (Node's Gini index). To the right of the tree, the Gini index formula is presented: $Gini(set) = \frac{P}{T} \left(1 - \frac{P}{T} \right) + \frac{N}{T} \left(1 - \frac{N}{T} \right)$. The formula is annotated with arrows and labels: 'Positive 데이터 갯수' (Positive data count) points to P ; 'Negative 데이터 갯수' (Negative data count) points to N ; '노드 내 전체 데이터 개수' (Total data count in node) points to T ; and '노드의 지니계수' (Node's Gini index) points to the entire formula.

Positive 데이터 갯수

Negative 데이터 갯수

$Gini(set) = \frac{P}{T} \left(1 - \frac{P}{T} \right) + \frac{N}{T} \left(1 - \frac{N}{T} \right)$

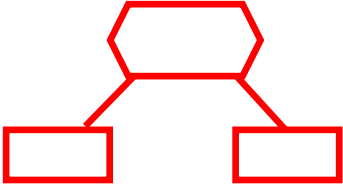
노드의 지니계수

노드 내 전체 데이터 개수

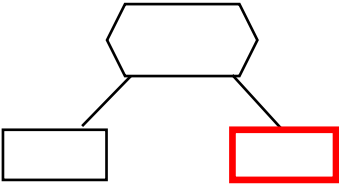
지니계수는 낮을 수록 좋음!

테스트 스코어

테스트 전체



각 노드의 지니계수


$$Q(Test) = \sum_{\substack{\text{sets} \\ \text{produced}}} D(sets) \times \frac{\text{해당 노드 데이터 수}}{\text{테스트 전체 데이터 수}}$$

노드 개수

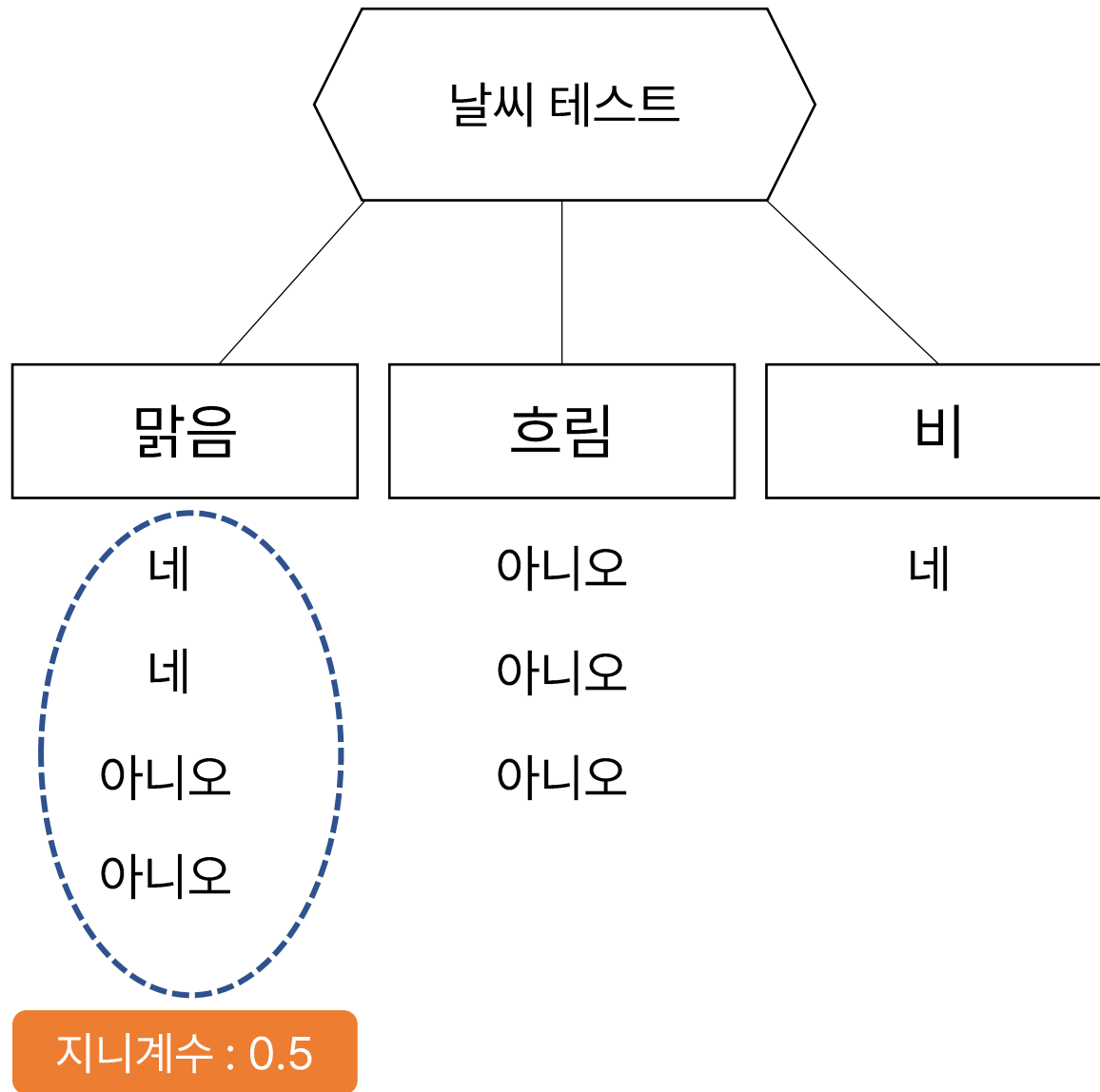
$$Gini(set) = \frac{P}{T} \left(1 - \frac{P}{T} \right) + \frac{N}{T} \left(1 - \frac{N}{T} \right)$$

$$T = 4$$

$$P = 2$$

$$N = 2$$

$$\begin{aligned} Gini(set) &= \frac{2}{4} \left(1 - \frac{2}{4} \right) + \frac{2}{4} \left(1 - \frac{2}{4} \right) \\ &= \frac{4}{16} + \frac{4}{16} \\ &= \frac{1}{2} \end{aligned}$$



$$Gini(set) = \frac{P}{T} \left(1 - \frac{P}{T} \right) + \frac{N}{T} \left(1 - \frac{N}{T} \right)$$

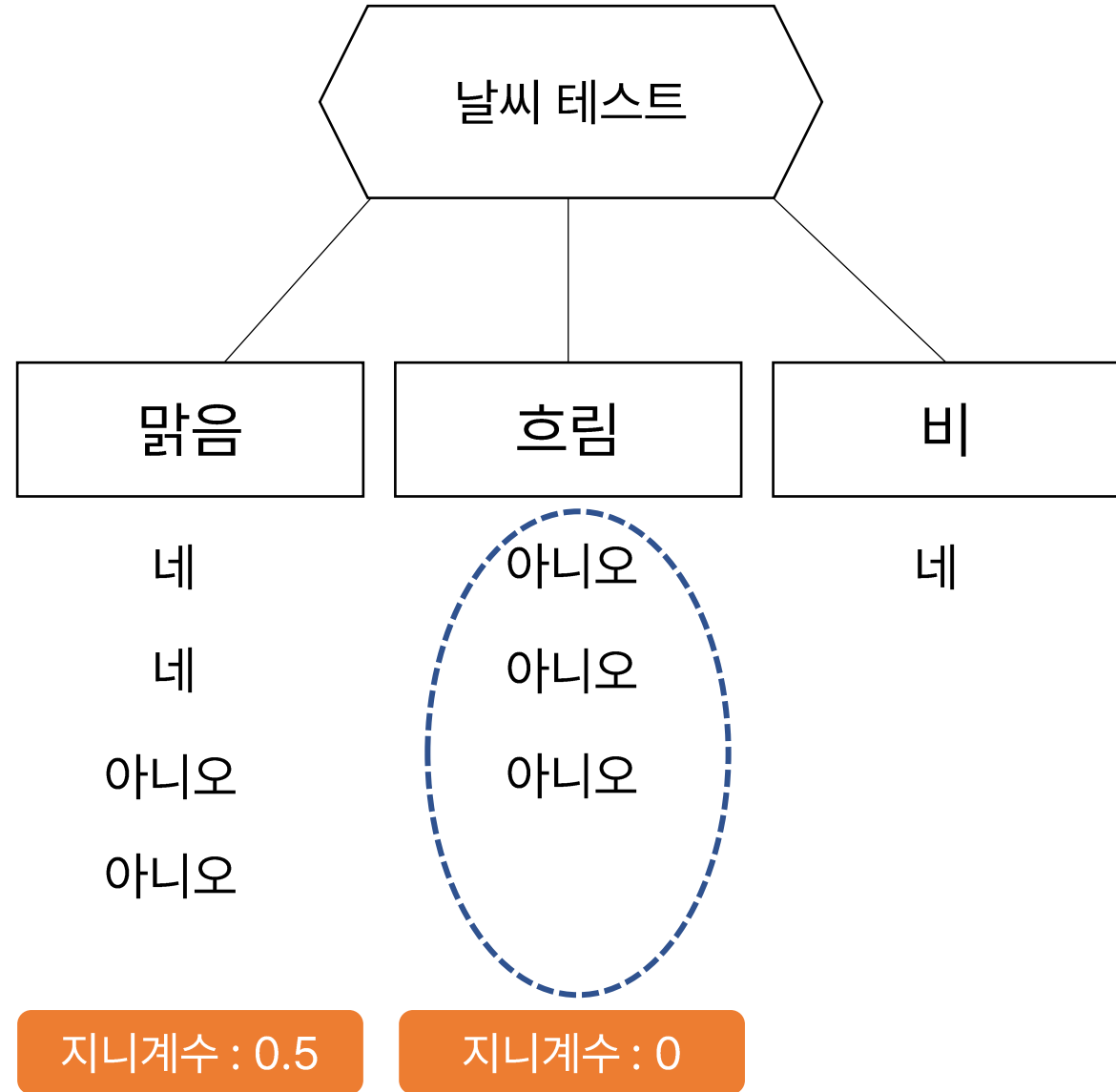
$$T = 3$$

$$P = 0$$

$$N = 3$$

$$Gini(set) = \frac{0}{3} \left(1 - \frac{0}{3} \right) + \frac{3}{3} \left(1 - \frac{3}{3} \right)$$

$$= 0$$



$$Gini(set) = \frac{P}{T} \left(1 - \frac{P}{T} \right) + \frac{N}{T} \left(1 - \frac{N}{T} \right)$$

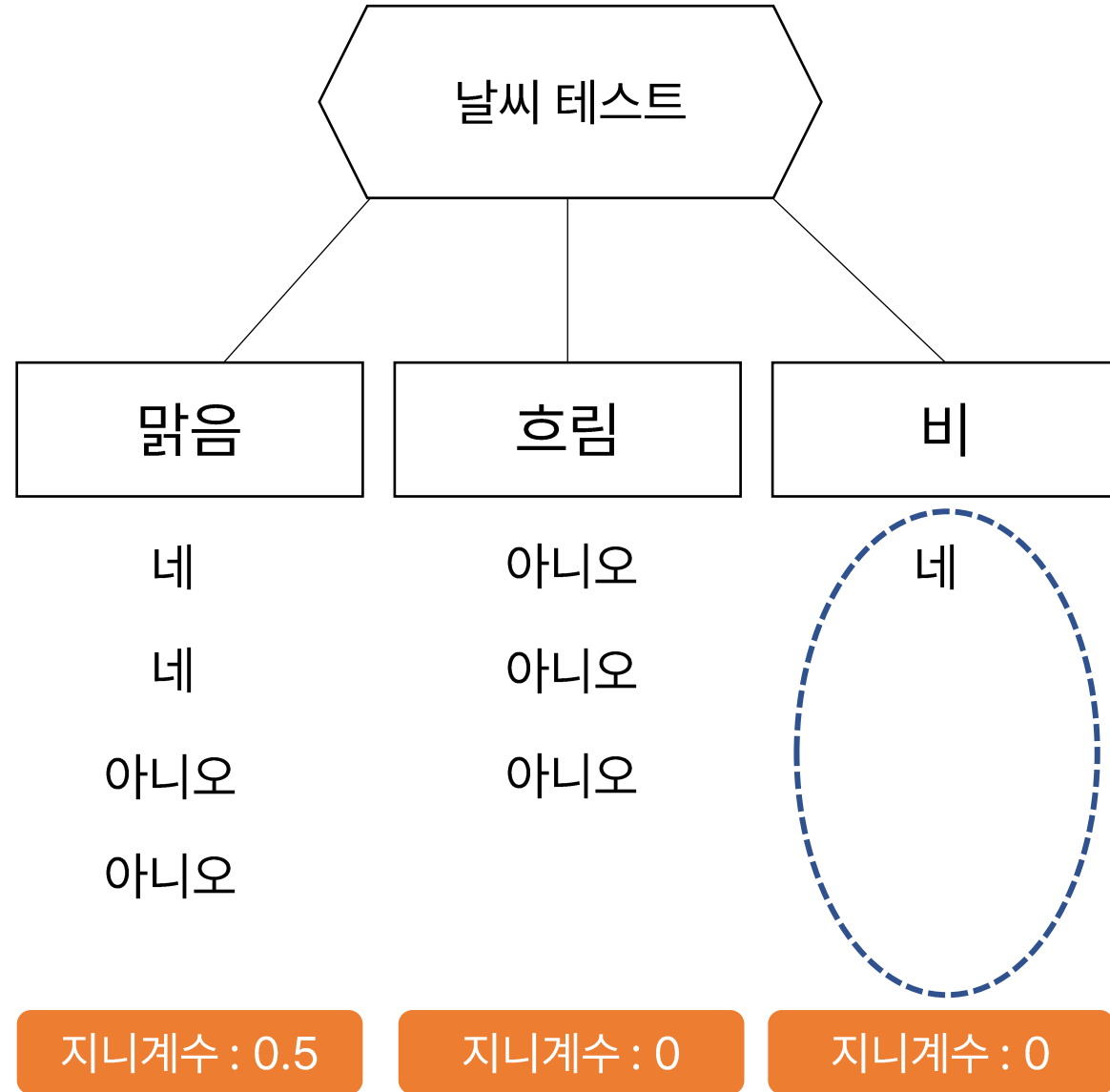
$$T = 1$$

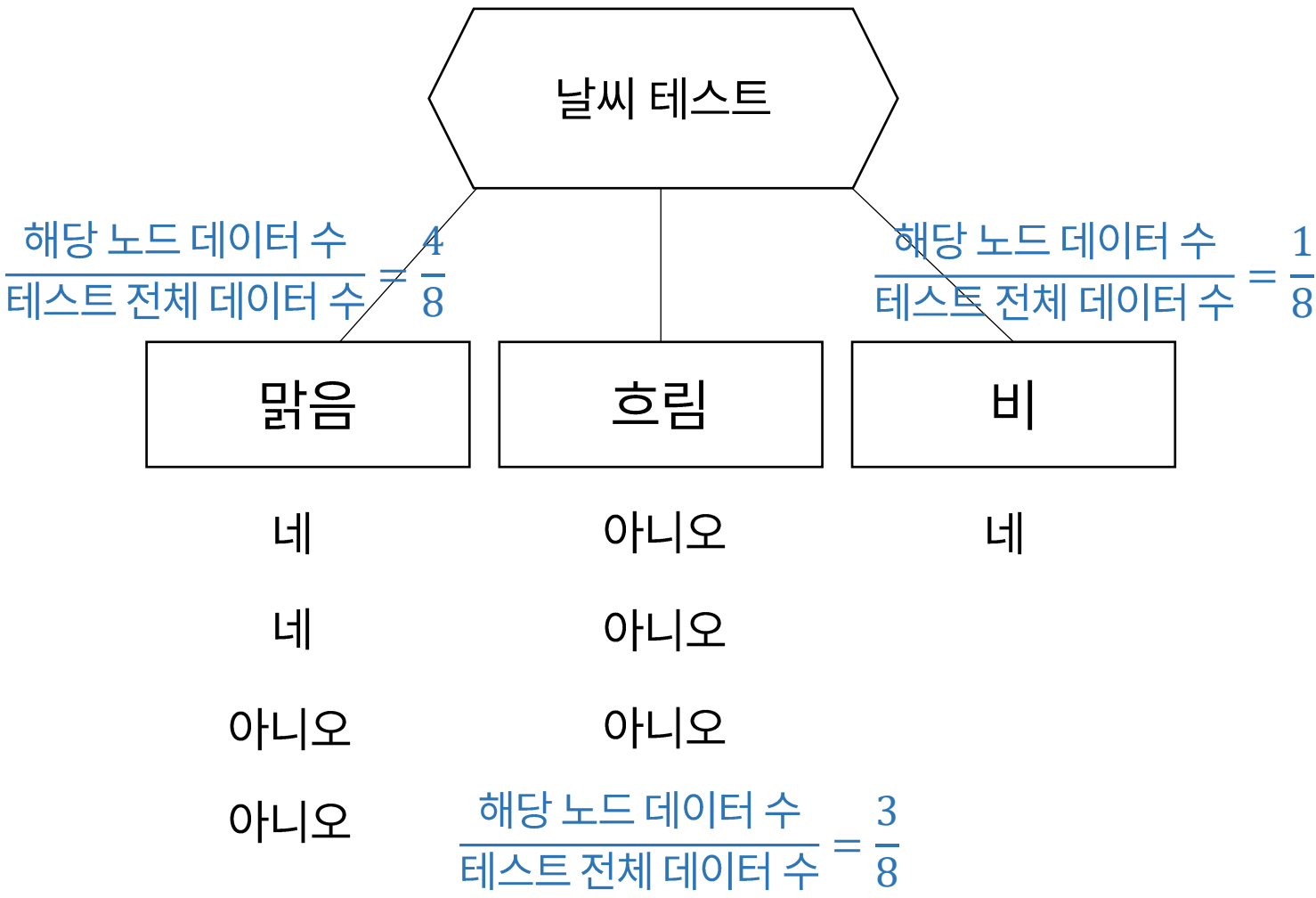
$$P = 1$$

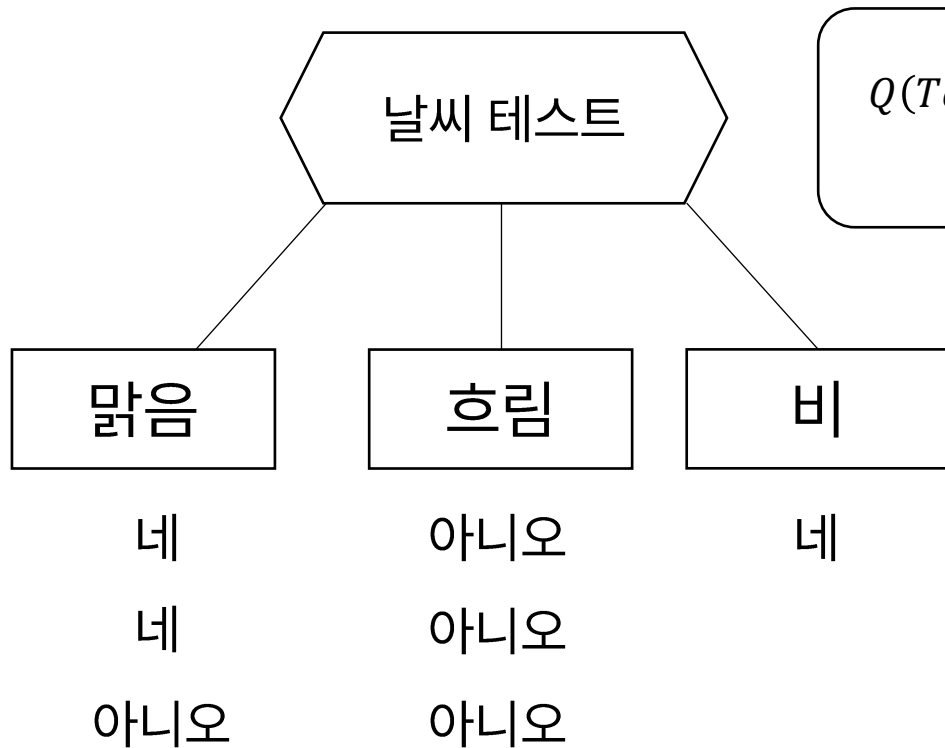
$$N = 0$$

$$Gini(set) = \frac{1}{1} \left(1 - \frac{1}{1} \right) + \frac{0}{1} \left(1 - \frac{0}{1} \right)$$

$$= 0$$

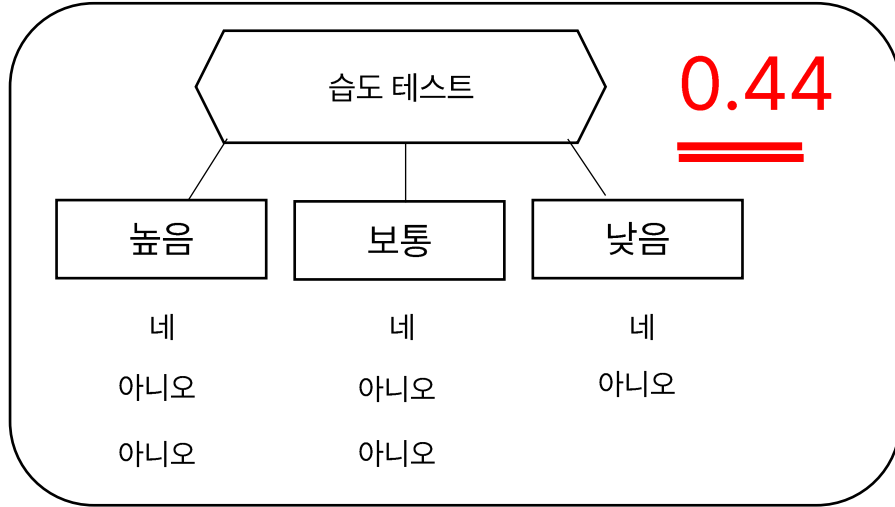
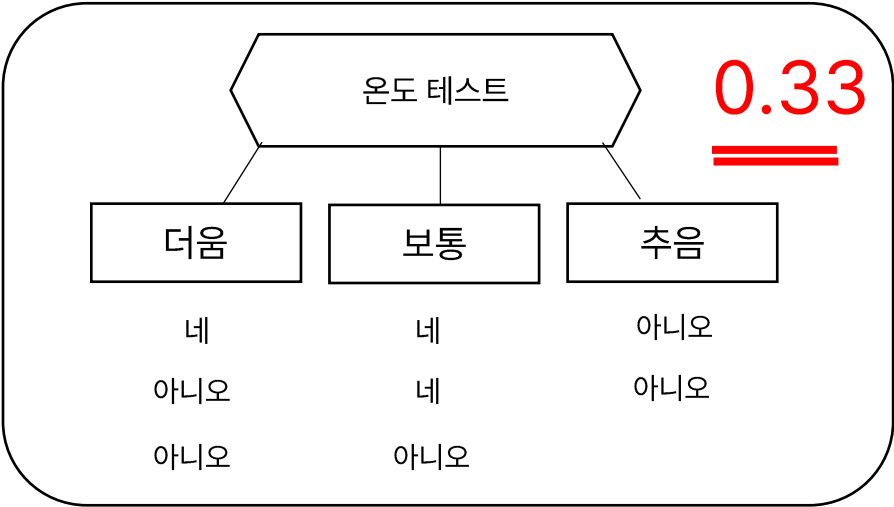
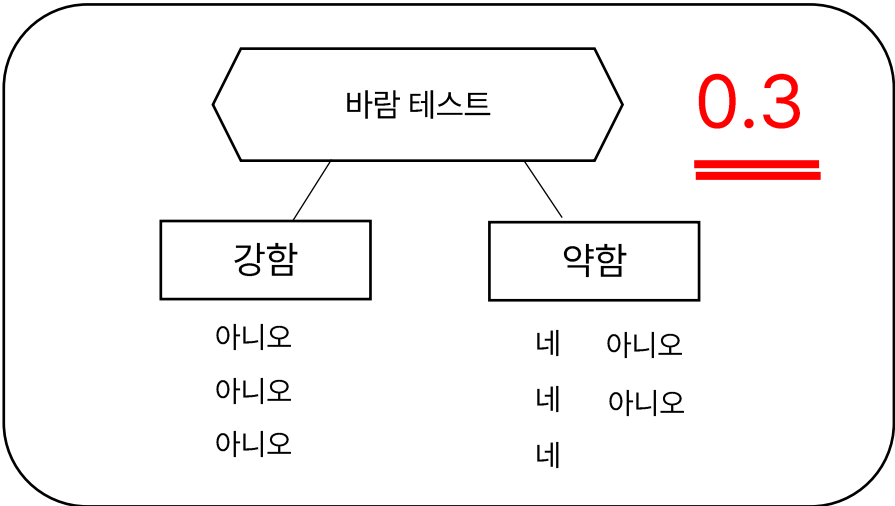
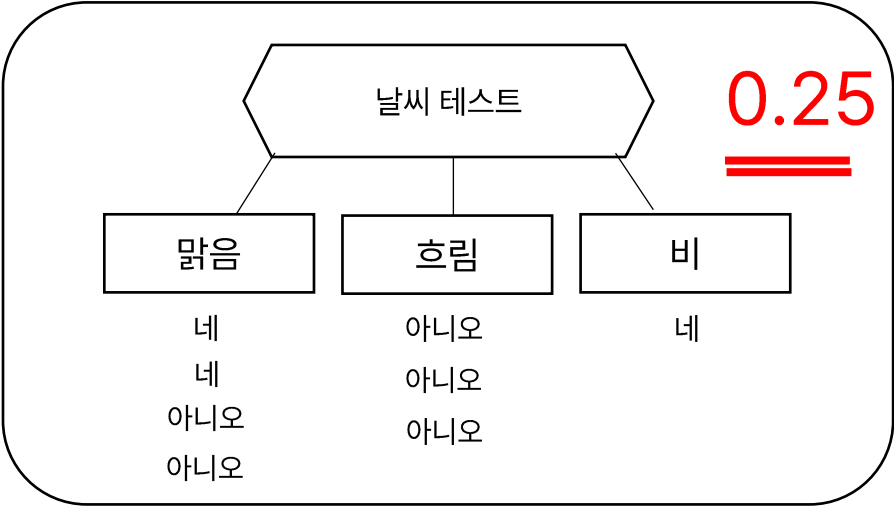


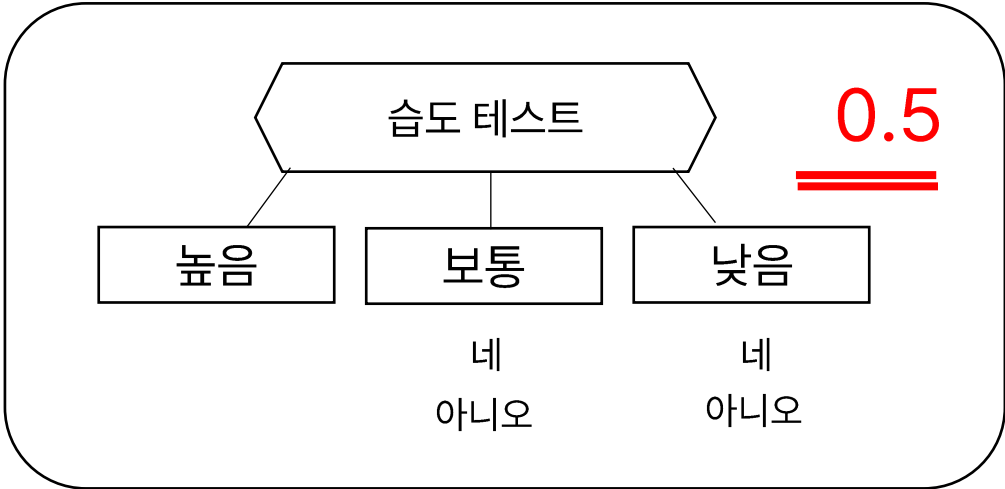
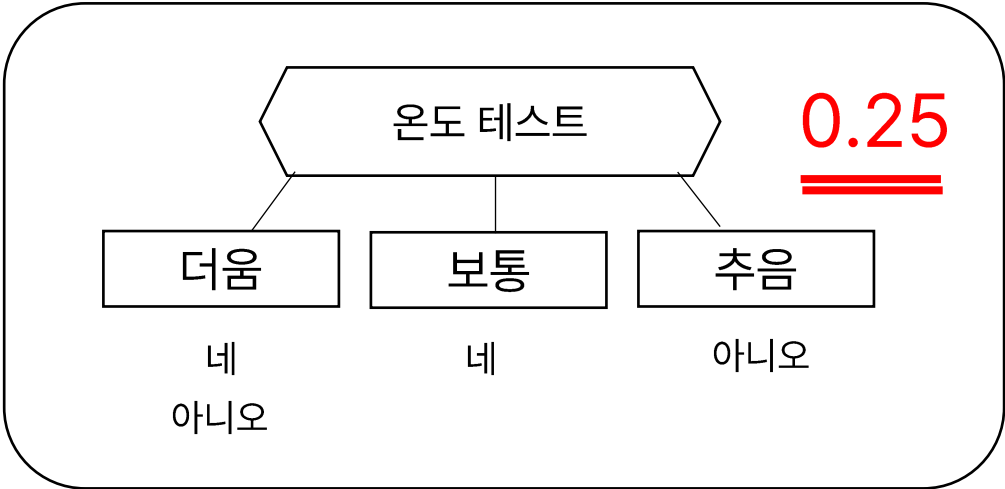
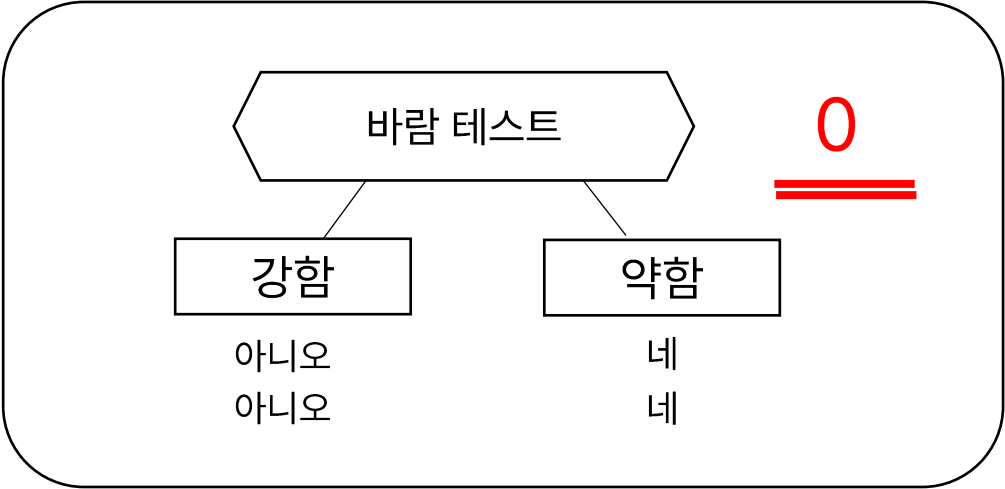


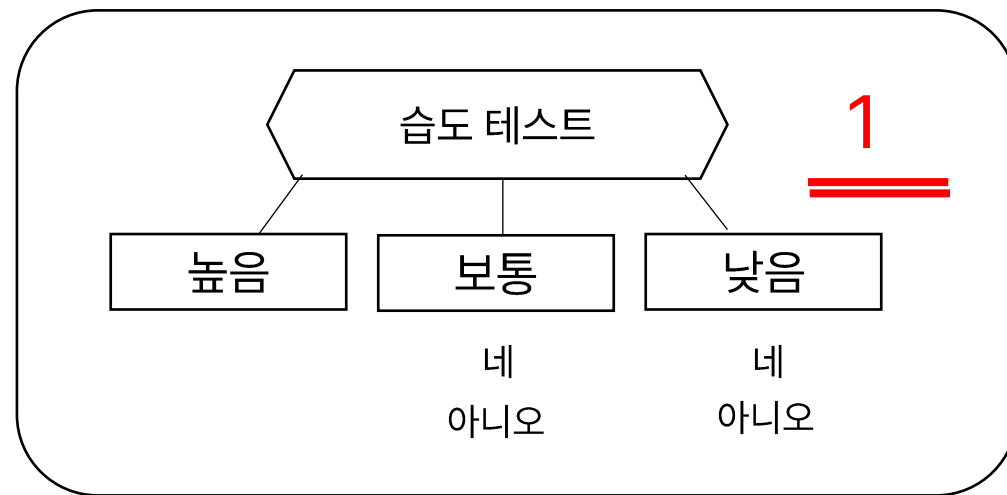
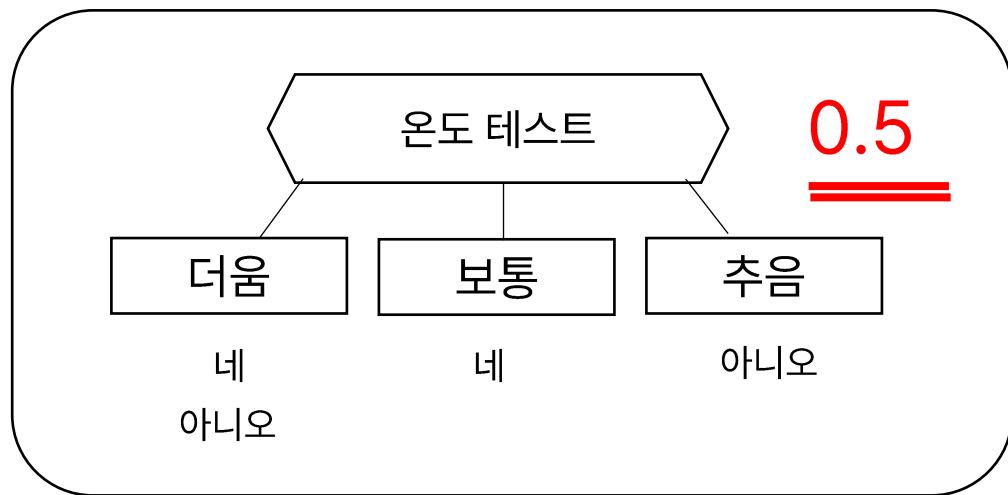
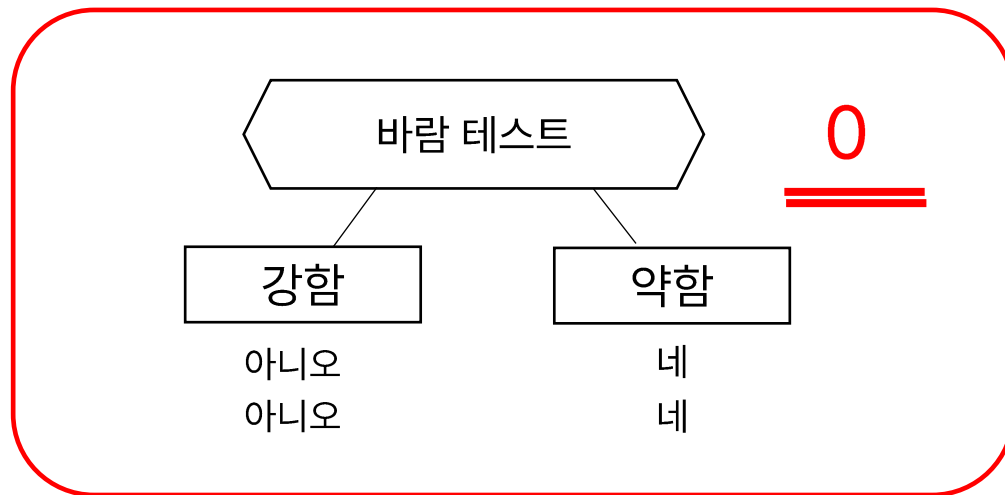


$$Q(Test) = \sum_{\text{sets produced}} Gini(\text{sets}) \times \frac{\text{해당 노드 데이터 수}}{\text{테스트 전체 데이터 수}}$$

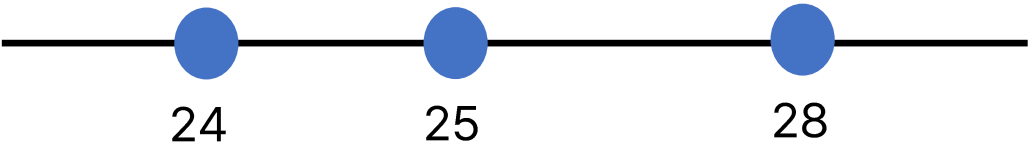
$$\begin{aligned}
 Q(Test) &= 0.5 \times \frac{4}{8} + 0 \times \frac{3}{8} + 0 \times \frac{1}{8} \\
 &= \frac{1}{4} + 0 + 0 \\
 &= \frac{1}{4}
 \end{aligned}$$



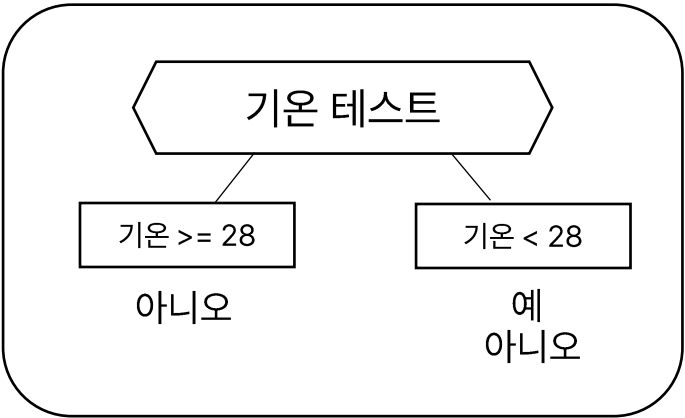
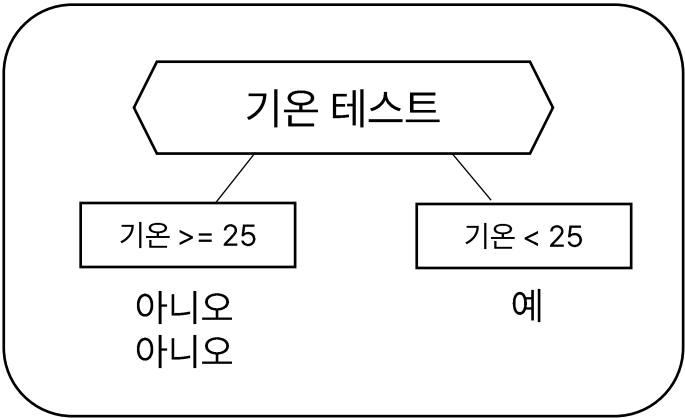
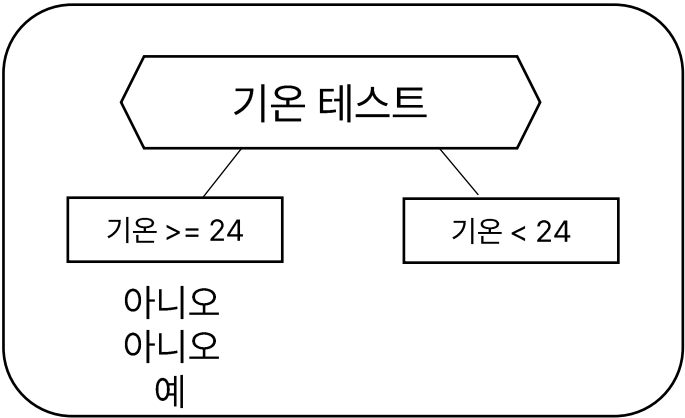




피처가 연속형일 경우: 모든 점이 테스트 후보



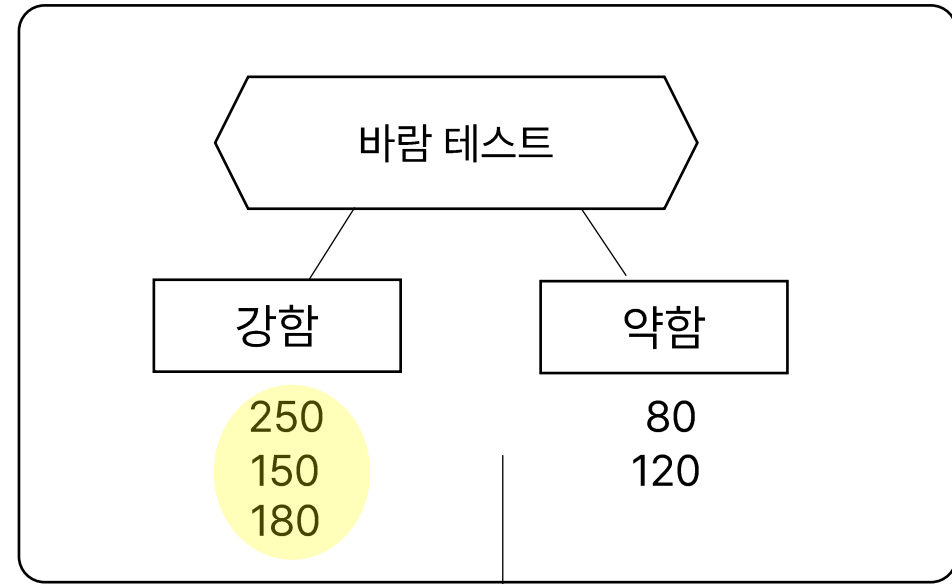
기온	배드민턴
25	아니오
28	아니오
24	예



타겟이 연속형일 경우: 평균값으로 추정

바람	배드민턴 상금
강함	250
약함	80
강함	150
약함	120
강함	180

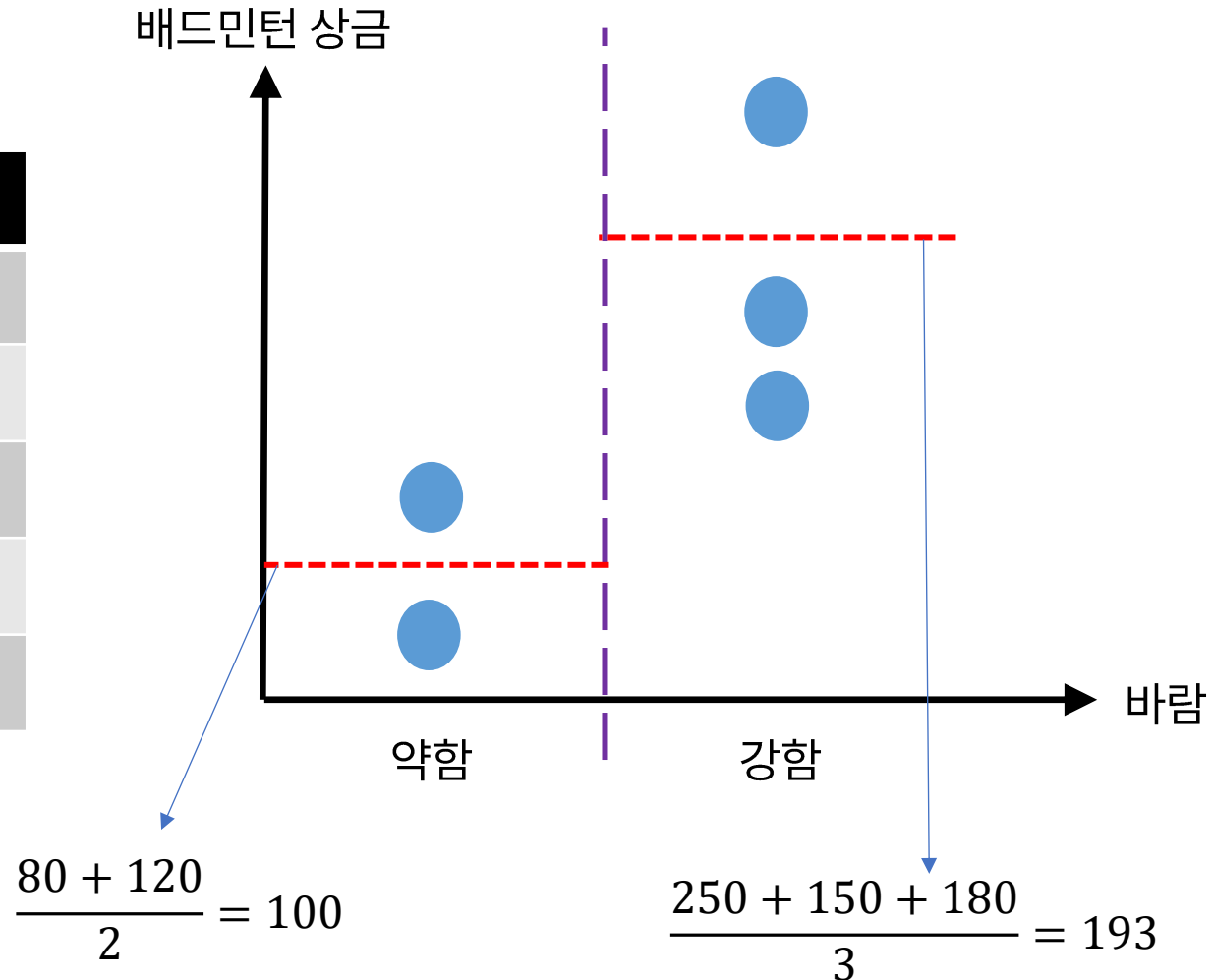
바람	배드민턴 상금
강함	193



$$\frac{250 + 150 + 180}{3} = 193$$

타겟이 연속형일 경우: 평균값으로 추정

바람	배드민턴 상금
강함	250
약함	80
강함	150
약함	120
강함	180



AI/BigData 인텐시브 과정

Section 6. 의사결정나무

Section 6-2. 의사결정나무 분류

의사결정나무 분류

```
In [1]: 1 import pandas as pd
        2 df = pd.read_csv("../data/wine_data.csv")
```

```
In [2]: 1 df
```

Out[2]:

Alcohol	Malic	Ash	Alcalinity	Magesium	Phenols	Flavanoids	Nonflavanoids	Proanthocyanins	Color	Hue	Dilution	Proline	class
14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065	0
13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050	0
13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185	0
14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480	0
13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
13.71	5.65	2.45	20.5	95	1.68	0.61	0.52	1.06	7.70	0.64	1.74	740	2
13.40	3.91	2.48	23.0	102	1.80	0.75	0.43	1.41	7.30	0.70	1.56	750	2
13.27	4.28	2.26	20.0	120	1.59	0.69	0.43	1.35	10.20	0.59	1.56	835	2
13.17	2.59	2.37	20.0	120	1.65	0.68	0.53	1.46	9.30	0.60	1.62	840	2
14.13	4.10	2.74	24.5	96	2.05	0.76	0.56	1.35	9.20	0.61	1.60	560	2

rows × 14 columns

의사결정나무 분류

```
In [3]: ▶ 1 features = ['Alcohol', 'Malic', 'Ash', 'Alcalinity', 'Magesium', 'Phenols', 'Flavanoids',  
2               'Nonflavanoids', 'Proanthocyanins', 'Color', 'Hue', 'Dilution', 'Proline']  
3  
4 X = df[features]  
5 y = df['class']
```

의사결정나무 분류

In [4]: ▶

```
1 # 트레이닝/테스트 데이터 분할
2 from sklearn.model_selection import train_test_split
3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

In [5]: ▶

```
1 #데이터 표준화
2 from sklearn.preprocessing import StandardScaler
3 std_scale = StandardScaler()
4 std_scale.fit(X_tn)
5 X_tn_std = std_scale.transform(X_tn)
6 X_te_std = std_scale.transform(X_te)
```


의사결정나무 분류

In [6]: ▶

```

1 # 의사결정나무 학습
2 from sklearn import tree
3 clf_tree = tree.DecisionTreeClassifier(random_state=0)
4 clf_tree.fit(X_tn_std, y_tn)

```

피처 성능이 동일한 경우 랜덤으로 선택

Out[6]:

```

▼ DecisionTreeClassifier
DecisionTreeClassifier(random_state=0)

```

In [7]: ▶

```

1 # 예측
2 pred_tree = clf_tree.predict(X_te_std)
3 print(pred_tree)

```

```

[0 2 1 0 1 1 0 2 1 1 2 2 0 1 2 1 0 0 2 0 1 0 1 1 1 1 1 1 2 0 0 1 0 0 0 2
 1 1 2 1 0 1 1 1]

```

의사결정나무 분류

```
In [8]: ▶ 1 # 정확도(accuracy)
2 from sklearn.metrics import accuracy_score
3 accuracy = accuracy_score(y_te, pred_tree)
4 print(accuracy)
```

0.9333333333333333

```
In [9]: ▶ 1 # 정밀도(precision)
2 from sklearn.metrics import precision_score
3 precision = precision_score(y_te, pred_tree, average='macro')
4 print(precision)
```

0.9326599326599326

```
In [10]: ▶ 1 # 리콜
2 from sklearn.metrics import recall_score
3 recall = recall_score(y_te, pred_tree, average='macro')
4 print(recall)
```

0.9424603174603176

```
In [11]: ▶ 1 # f1 score
2 from sklearn.metrics import f1_score
3 f1 = f1_score(y_te, pred_tree, average='macro')
4 print(f1)
```

0.9349141206870346

의사결정나무 분류

In [12]: ▶

```
1 # confusion matrix 확인
2 from sklearn.metrics import confusion_matrix
3 conf_matrix = confusion_matrix(y_te, pred_tree)
4 print(conf_matrix)
```

```
[[14  2  0]
 [ 0 20  1]
 [ 0  0  8]]
```

In [13]: ▶

```
1 # 분류 레포트 확인
2 from sklearn.metrics import classification_report
3 class_report = classification_report(y_te, pred_tree)
4 print(class_report)
```

	precision	recall	f1-score	support
0	1.00	0.88	0.93	16
1	0.91	0.95	0.93	21
2	0.89	1.00	0.94	8
accuracy			0.93	45
macro avg	0.93	0.94	0.93	45
weighted avg	0.94	0.93	0.93	45

AI/BigData 인텐시브 과정

Section 6. 의사결정나무

Section 6-3. 의사결정나무 회귀

의사결정나무 회귀

In [14]: ▶

1

import pandas as pd

2

df = pd.read_csv("./data/house_prices.csv")

In [15]: ▶

1

df

Out[15]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67	22.4
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08	20.6
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64	23.9
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48	22.0
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88	11.9

506 rows × 14 columns

의사결정나무 회귀

```
In [16]: ▶ 1 df['MEDV'].unique()
```

```
Out[16]: array([24. , 21.6, 34.7, 33.4, 36.2, 28.7, 22.9, 27.1, 16.5, 18.9, 15. ,
                21.7, 20.4, 18.2, 19.9, 23.1, 17.5, 20.2, 13.6, 19.6, 15.2, 14.5,
                15.6, 13.9, 16.6, 14.8, 18.4, 21. , 12.7, 13.2, 13.1, 13.5, 20. ,
                24.7, 30.8, 34.9, 26.6, 25.3, 21.2, 19.3, 14.4, 19.4, 19.7, 20.5,
                25. , 23.4, 35.4, 31.6, 23.3, 18.7, 16. , 22.2, 33. , 23.5, 22. ,
                17.4, 20.9, 24.2, 22.8, 24.1, 21.4, 20.8, 20.3, 28. , 23.9, 24.8,
                22.5, 23.6, 22.6, 20.6, 28.4, 38.7, 43.8, 33.2, 27.5, 26.5, 18.6,
                20.1, 19.5, 19.8, 18.8, 18.5, 18.3, 19.2, 17.3, 15.7, 16.2, 18. ,
                14.3, 23. , 18.1, 17.1, 13.3, 17.8, 14. , 13.4, 11.8, 13.8, 14.6,
                15.4, 21.5, 15.3, 17. , 41.3, 24.3, 27. , 50. , 22.7, 23.8, 22.3,
                19.1, 29.4, 23.2, 24.6, 29.9, 37.2, 39.8, 37.9, 32.5, 26.4, 29.6,
                32. , 29.8, 37. , 30.5, 36.4, 31.1, 29.1, 33.3, 30.3, 34.6, 32.9,
                42.3, 48.5, 24.4, 22.4, 28.1, 23.7, 26.7, 30.1, 44.8, 37.6, 46.7,
                31.5, 31.7, 41.7, 48.3, 29. , 25.1, 17.6, 24.5, 26.2, 42.8, 21.9,
                44. , 36. , 33.8, 43.1, 48.8, 31. , 36.5, 30.7, 43.5, 20.7, 21.1,
                25.2, 35.2, 32.4, 33.1, 35.1, 45.4, 46. , 32.2, 28.5, 37.3, 27.9,
                28.6, 36.1, 28.2, 16.1, 22.1, 19. , 32.7, 31.2, 17.2, 16.8, 10.2,
                10.4, 10.9, 11.3, 12.3,  8.8,  7.2, 10.5,  7.4, 11.5, 15.1,  9.7,
                12.5,  8.5,  5. ,  6.3,  5.6, 12.1,  8.3, 11.9, 17.9, 16.3,  7. ,
                7.5,  8.4, 16.7, 14.2, 11.7, 11. ,  9.5, 14.1,  9.6,  8.7, 12.8,
                10.8, 14.9, 12.6, 13. , 16.4, 17.7, 12. , 21.8,  8.1])
```

Section

의사결정나무 회귀

In [17]: 1 df.columns

Out[17]: Index(['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX',
'PTRATIO', 'B', 'LSTAT'],
dtype='object')

In [18]: 1 features = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT']
2
3 X = df[features]
4 y = df['MEDV']

In [19]: 1 X

Out[19]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33
...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88

506 rows × 13 columns

의사결정나무 회귀

In [20]: ▶

```
1 # 트레이닝/테스트 데이터 분할
2 from sklearn.model_selection import train_test_split
3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

In [21]: ▶

```
1 #데이터 표준화
2 from sklearn.preprocessing import StandardScaler
3 std_scale = StandardScaler()
4 std_scale.fit(X_tn)
5 X_tn_std = std_scale.transform(X_tn)
6 X_te_std = std_scale.transform(X_te)
```


Section

의사결정나무 회귀

의사결정나무 회귀

In [22]: ▶

```
1 # 의사결정나무 학습
2 from sklearn.tree import DecisionTreeRegressor
3 clf_tree = DecisionTreeRegressor(random_state=0)
4 clf_tree.fit(X_tn_std, y_tn)
```

Out[22]:

```
▼ DecisionTreeRegressor
DecisionTreeRegressor(random_state=0)
```

In [23]: ▶

```
1 # 예측
2 tree_pred = clf_tree.predict(X_te_std)
3 print(tree_pred)
```

```
[23.7 25.  22.7 16.3 22.  20.4 20.6 19.  15.3 16.1 10.5 17.9 12.6  8.8
 50.  37.  21.4 34.9 22.8 19.6 24.7 22.3 20.1 24.7 22.5 10.9 14.5 15.6
 39.8 18.8 12.1 19.1 25.  22.2 22.  22.6 10.2 22.6 12.1 17.9 23.9 23.2
 22.  11.7 26.4 19.  22.5 13.6 13.8 27.5 13.1 19.7 24.  35.2 13.9 18.9
 19.4 17.6 10.9 22.4 20.6 22.5 29.6 22.8 23.2 24.  18.4 18.9 10.4 22.5
 20.8 22.2 24.1 30.1 25.   8.8 41.7 23.1 22.  24.5 27.1 14.5 21.4 37.6
 37.6 25.  20.7 13.8 24.4 16.1 16.6 11.8 20.6 30.1 21.1 23.8 10.2 22.9
 12.7 18.5 24.7 20.1 30.7 24.5 21.  18.9  7.  15.3 23.9 22.  37.  17.9
 18.9 19.7 18.9 20.8  7.5 19.2 10.2 50.  30.1  6.3 14.4 21.4 25.  20.5
 36.1]
```

In [24]: ▶

```
1 # 모형 평가-MSE
2 from sklearn.metrics import mean_squared_error
3 mse = mean_squared_error(y_te, tree_pred)
4 print(mse)
```

```
27.977480314960626
```

감사합니다.

Q & A